Developer-on-the-Loop: Engineering Cognitive Systems with Coding Agents

Robert E. Wray ROBERT.WRAY@CIC.IQMRI.ORG
Center for Integrated Cognition, IQM Research Institute, Ann Arbor, MI 48105 USA

Moritz Schmidt MORITZ.SCHMIDT@THA.DE
Technical University of Applied Sciences Augsburg, Faculty of Electrical Engineering, Germany

Steven J. Jones STEVEN.JONES@CIC.IQMRI.ORG
Center for Integrated Cognition, IQM Research Institute, Ann Arbor, MI 48105 USA

Abstract

Developing and maintaining cognitive-architecture-based agents remains labor-intensive and requires significant expertise, constraining both the scale of deployed systems and the breadth of cognitive-systems research. Research has shown that LLM-based coding agents can generate instances of knowledge encodings for cognitive systems. We build on this prior work to explore if (and to what extent) coding agents can support various stages undertaken during the creation of novel cognitive systems. We first introduce a framework that characterizes the cognitive-systems engineering lifecycle and then evaluate the use of coding agents across various phases of the lifecycle, presenting implemented case studies for Soar-based cognitive systems. Across case studies spanning specification, implementation, testing, debugging, refactoring, and architecture maintenance, effective use of coding agents required more explicit, *a priori* specification of requirements, representations, and design assumptions than is typical in cognitive-systems development; this process also surfaced implicit design commitments and supports improved maintainability. The human role thus shifted from hands-on implementation toward specification, supervision, and validation (“on the loop"), suggesting that the long-term contribution of coding agents may lie less in reducing implementation effort than in maturing cognitive-systems engineering practice itself.

1. Introduction

Cognitive systems researchers often emphasize designing, implementing, and refining integrated cognitive mechanisms and representations, or cognitive architectures (Kotseruba & Tsotsos, 2025; Langley et al., 2009). However, to demonstrate and to evaluate architectures, they must also create agents or applications in various domains (Newell, 1990). The development of such cognitive systems has historically required substantial knowledge engineering effort: encoding domain knowledge, constructing task-specific procedures, diagnosing failures, and maintaining complex

code bases.[1] Across architectures, this work generally remains labor-intensive and requires specialized expertise. As a result, the cost of developing and maintaining cognitive agents has often limited both the scale of deployed systems and the breadth of experimental investigations.

Recent advances in Large Language Models (LLMs), particularly models optimized for software development tasks, have led to significant disruption in traditional software development (Weisz et al., 2025; Panyam & Gujar, 2025). We use the term *coding agent* to capture the interactive assistants that use LLMs in the background, often embedded within integrated development environments (IDEs). Examples include Claude Code, GitHub Copilot, OpenCode, Goose, etc. For traditional software engineering tasks, coding agents can generate source code, diagnose software defects, propose refactorings, and use external tools (e.g., commit code changes to a repository).

To what extent can coding agents support development of cognitive-architecture-based agents? Just as LLMs can contribute to the knowledge dynamically accessible to a cognitive agent (e.g., Nakos & Forbus, 2025; Kirk et al., 2024), we can also investigate their applicability in supporting various cognitive-systems development processes. At first glance, the application of coding agents to cognitive architectures may appear straightforward. The knowledge representations of cognitive architectures that support symbolic systems directly are ultimately expressed in programming languages or architecture-specific representations that should be amenable to code generation. However, cognitive-systems development differs from conventional software engineering in several important aspects. Developers must reason about symbolic representations and the underlying cognitive processes and architectural constraints. Further, the functional requirements for a cognitive system are often challenging to define completely or explicitly in advance. Consequently, it is not obvious which (if any) aspects of development can be delegated to coding agents and which continue to require substantial human expertise.

This paper examines the role of coding agents for cognitive agents implemented with Soar (Laird, 2012). To support this investigation, we developed tooling that enabled coding agents to execute Soar models, inspect outputs, diagnose failures, and iteratively revise agent implementations. We then explored the use of coding agents across a range of development activities, including production rule generation, operator implementation, debugging, testing, and refactoring. These activities provide an empirical basis for evaluating both the capabilities and limitations of coding-agent-assisted development within a cognitive architecture context.

Our findings suggest that coding agents may not only automate some aspects of agent knowledge encoding, but could help transform the development process itself. Researchers have recently concluded that similar transformation may be underway in software engineering more broadly (Hoda, 2026). Effective collaboration with coding agents required substantially more explicit specification of requirements, representations, and design assumptions than typical cognitive-systems research processes. However, the process of defining specifications often clarified unresolved design decisions and exposed implicit assumptions about agent behavior. Thus, coding agents may function not only as implementation assistants but also as a junior partner in knowledge engineering, shifting a developer's role from direct construction toward specification, supervision, and validation,

1. Increasingly, traditional knowledge engineering is complemented or supplemented by training processes, such as policy learning or explicit instruction. While these processes differ in detail from traditional knowledge engineering, they still usually require substantial human design and implementation efforts.

especially for domain-specific applications or agents (cognitive-systems engineering). The implications of this shift extend beyond the specific use cases and examples in this paper, suggesting new approaches to the development and maintenance of cognitive architecture-based systems.

2. Constructing Larger-Scale Cognitive Systems

Since the emergence of cognitive architectures, researchers have recognized that progress depends not only on advances in architectural mechanisms but also on the ability to construct, maintain, and evaluate increasingly complex cognitive systems (Newell, 1990). Importantly, as early architectures matured and began to be used as a foundation for applications (e.g., Jones et al., 1999; Gray et al., 1995; Corbett et al., 2001), some attention in the community shifted toward the engineering challenges associated with building larger-scale, more capable cognitive agents and testing them, often in challenging, simulation-based environments (Pew & Mavor, 1998; Laird, 2001; Schoelles & Gray, 2001; Gray, 2002; Gluck & Pew, 2005).

Methods and practices that can support the systematic development of cognitive systems, what might broadly be described as *cognitive-systems engineering*,[2] span the development of methodologies, tools, and processes that improve the construction of cognitive agents, including knowledge acquisition, representation design, debugging, validation, and long-term maintenance. Cognitive-systems researchers have explored a variety of approaches, including software engineering methods for cognitive modeling (Salvucci, 2016, e.g.,), integrated specification and development environments (John et al., 2004; Koedinger et al., 2004), compilation from abstract or high-level languages (Crossman et al., 2004; Ritter et al., 2006; Salvucci & Lee, 2003; Jones et al., 2006; St. Amant et al., 2006), model tracing and debugging tools (Taylor et al., 2002, 2006; MacLellan et al., 2014), knowledge engineering methodologies (Schreiber et al., 2000), and reusable libraries of cognitive components (Taylor & Wray, 2004). For example, work associated with ACT-R has often emphasized model development and evaluation methodologies, while Soar researchers developed tools for trace analysis, explanation, and agent debugging. While details differ, a key objective of cognitive-systems engineering broadly is to reduce the cost and difficulty of developing cognitive systems while improving their reliability and maintainability.

A recurring theme across this work is that cognitive-system capability is often constrained as much by engineering factors (expertise, time, cost, etc.) as by architectural limitations. The construction of cognitive agents often requires tedious manual effort to translate informal requirements into explicit representations, procedures, and knowledge structures. As systems grow in complexity, the burden of maintaining consistency among these artifacts can become a significant obstacle to further development and undercuts the ability to "cumulate" results over time, a key factor in the original conception of cognitive-architecture evaluation (Newell, 1990). The emergence of coding agents introduces a new possibility. Rather than merely providing better tools for human developers, coding agents may participate directly in the development process itself, generating, testing, debugging, and revising cognitive systems under human supervision. This raises the question addressed

2. We mean principles and tools that define an efficient and reproducible methodology for the development of cognitive systems, in contrast to Cognitive Systems Engineering (Hollnagel & Woods, 1983) in Human Factors.

in this paper: **do coding agents represent another incremental improvement in development tooling or offer more fundamental shift in the practice of engineering cognitive systems?**

A few recent efforts have explored direct integration of LLMs to assist in the generation of cognitive-agent code. They provide proof-of-concept demonstrations that motivate our approach. Wu et al. (2023) investigated the use of prompt engineering to support the generation of ACT-R and Soar model components, demonstrating that contemporary language models can produce architecture-specific artifacts with appropriate guidance. Similarly, Zhu & Simmons (2024) explored the generation of Soar productions from natural language task descriptions, providing evidence that language models can assist in translating procedural knowledge into executable rules. Wray et al. (2025) evaluate the use of a sequential series of LLM calls to construct and refine productions that implement a complete Soar problem space. These approaches use LLM code generation without a feedback loop. Yuan et al. (2025) combine Soar production generation with limited *in situ* testing of generated rules, moving closer to the (semi-)autonomous workflows we investigate.

In contrast to direct LLM generation, recent advances in LLMs have produced a class of systems commonly referred to as *coding agents*: software systems built around LLMs capable of generating software artifacts, interpreting source code, diagnosing failures, and interacting with external development tools. Unlike earlier code-generation systems, modern coding agents can operate within iterative development loops in which they document requirements, produce specifications, generate code, execute tests, observe failures, and revise their outputs in response to execution feedback. These capabilities have led to widespread adoption in software engineering tasks including implementation, debugging, refactoring, documentation, and test generation.

This work extends prior work in two ways. First, rather than focusing exclusively on using LLMs to encode knowledge, we examine a broader range of activities that arise during cognitive-systems engineering, including specification, design, debugging, and refactoring, using coding agents to support these tasks rather than LLMs alone. Second, we use coding agents integrated within IDEs, thus permitting coding agents to directly inspect, execute, diagnose, and revise cognitive systems under development. This broader perspective allows us to investigate not only whether coding agents can generate cognitive-systems artifacts, but how they may alter the overall process by which cognitive systems are engineered.

3. Potential Roles for Coding Agents in Cognitive-Systems Engineering

This section first introduces a framework that outlines the cognitive-systems engineering lifecycle, contrasting cognitive-systems development with traditional software engineering (SE) for each phase. We then consider the potential role of coding agents in supporting the cognitive-engineering process, again emphasizing the unique requirements imposed by cognitive systems.

3.1 Framework for Cognitive-Systems Engineering

Figure 1 summarizes a cognitive-systems-engineering lifecycle, identifying the span of activities in that lifecycle. The cognitive-systems lifecycle includes research and development of cognitive architecture (gray boxes) and the development of cognitive-systems demonstrations/applications (colored boxes) built on these architectures. Cognitive architecture R&D involves investigating and

refining the capabilities of a cognitive architecture. The cognitive-systems development process involves the creation of applications or demonstrations of a cognitive agent in a particular domain, using the architecture.

A distinct challenge for CSE, in comparison to SE, is the highly iterative "inner loop" of cognitive systems development, which is omitted in the figure for legibility. Although both SE and CSE follow similar, abstract "waterfall" progressions as in the figure, in practice, even in SE, there are many retreats to prior steps (redesign, a bug that requires re-implementation of an algorithm, etc.). Cognitive-systems engineering often requires many more design–implement–test iterations than SE, as well as often returning to the requirements stage as well. This iteration results from the challenge of modeling heuristic knowledge rather than rigid and well-specified procedures typical of SE (see, e.g., Studer et al., 1998). These challenges have led to various, distinct design methodologies in both knowledge-based systems (Schreiber et al., 2000) and cognitive systems (Crossman et al., 2004) to address this challenge.

Another unique challenge of cognitive-systems development, in comparison to traditional software engineering, is that application development is dependent on the cognitive architecture, but the development processes of each are often ongoing. This parallelism is comparable to developing software to execute on a hardware architecture or operating system that is not yet fully developed. Practically, most cognitive-systems development occurs during periods when the architecture is largely stable but, in comparison to traditional software engineering assumptions, the computational substrate on which development occurs is both changing more frequently and with less conservatism with respect to backwards compatibility.

Below, we outline these phases, focusing particularly on the development of cognitive systems (top process), which has greater commonality than architectural development across the cognitive-systems community. We contrast with SE where notable and illustrate with specific Soar examples. The phases are intended to suggest major classes of cognitive-systems engineering activities rather than to prescribe a sequential development process; as above, and in our own direct experience, these steps are often iterative with many loops and jumps. Finally, we omit steps that are more relevant to research than the cognitive system itself, such as the creation of evaluation scenarios or data analysis.

Identify and Document System Requirements: Requirements analysis establishes the capabilities, constraints, performance objectives, and operating environments that the cognitive system must satisfy. Artifacts produced during this stage may include requirements documents, use cases, operational scenarios, domain analyses, and descriptions of relevant tasks and environments. While requirements elicitation is common to both cognitive-systems engineering and SE, cognitive systems often operate in open-ended domains where desired behaviors cannot be completely specified in advance and reasoning, adaptation, or decision quality criteria may be specified rather than functional outcomes. Methods such as cognitive task analysis (Militello & Hutton, 1998; Schraagen et al., 2000) are often used to elicit requirements from domain experts for cognitive systems.

Create System Specification: System specification translates requirements into a detailed description of the system to be constructed. Artifacts may include interface specifications, representation designs, procedural descriptions, knowledge models, behavioral expectations, and acceptance criteria. In contrast to SE, where specifications often focus on data structures, algorithms, and

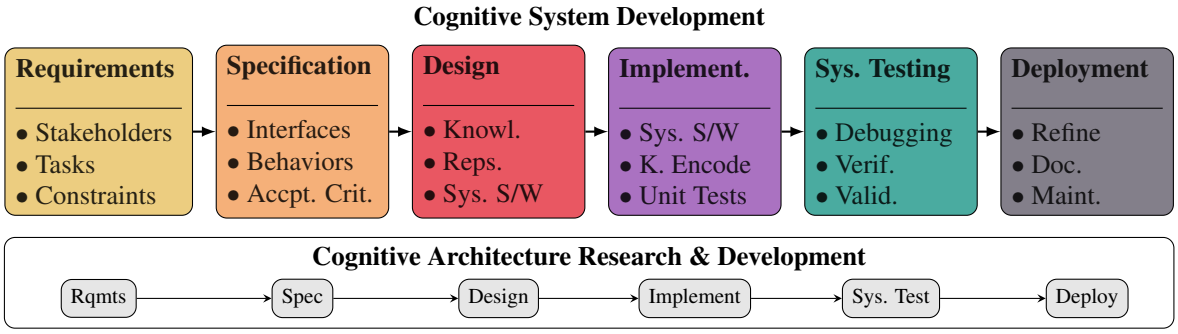


Figure 1: Cognitive-systems engineering activities, divided into cognitive-architecture R&D (bottom) and development of cognitive systems/applications (top). Tasks shown within boxes are representative, not exhaustive. Each stage is revisited many times during development (loops omitted).

component interactions, cognitive-systems engineering must additionally specify the knowledge, reasoning processes, and behavioral competencies required to achieve intelligent behavior. Specific tools for coding agents (spec.io) have been developed to support specification in SE.

Design Cognitive System: During design, developers determine how the cognitive system will be organized to satisfy the system specification. As in SE, this step includes systems-software design such as defining system components, interfaces, middleware, and interactions with external systems. Cognitive-systems engineering additionally requires the design of the agent's knowledge organization and representations, including how information is encoded, structured, and manipulated by the underlying cognitive architecture. For example, for Soar agents, design typically spans working-memory organization, an initial problem space decomposition (operators, impasses, and default knowledge used for impasses), and commitments to specific knowledge representations.

Implementation: Developers construct software components and encode knowledge identified in system design. While SE focuses primarily on implementing algorithms and data structures, cognitive-systems engineering additionally requires knowledge engineering: encoding procedural, semantic, and other forms of knowledge in representations appropriate for the underlying cognitive architecture. In Soar, implementation involves coding productions, initializing semantic memory, and refining knowledge content and organization incrementally (in response to both ongoing development and agent learning). Unit tests are also developed during this stage. Unlike SE, where tests are often derived directly from detailed specifications, many subtleties of cognitive-systems behaviors often only emerge through iterative development, requiring concurrent test construction.

Systems Testing and Debugging: Testing evaluates whether the implemented cognitive system satisfies its functional and behavioral requirements. As in SE, this step includes integration and system-level testing to identify implementation defects and verify correct operation. However, because cognitive systems operate in complex, open-ended domains where detailed behavioral requirements are difficult to specify fully in advance, testing frequently reveals deficiencies not only in the implementation but also in the underlying knowledge representations, reasoning strategies, or even the original system specification. Consequently, testing and debugging often drive substantial

Step	Potential Role of Coding Agent
Requirements	Organize stakeholder inputs, generate draft requirements documents, identify ambiguities or inconsistencies, and maintain traceability.
Specification	Generate **detailed, structured specifications from informal descriptions**, identify under-specified aspects of design.
Design	Transform specifications into **detailed design artifacts**. Maintain consistency among evolving design artifacts.
Implementation	Generate code for both software components and **cognitive knowledge encoded in architecture-specific representations**. Generate/update unit tests as the system evolves.
Sys. Testing	Generate behavioral & unit tests, **analyze execution traces**, identify inconsistencies between expected & observed behavior. Propose revisions to s/w or knowledge artifacts.
Deployment	Refactor software and knowledge artifacts, **improve run-time efficiency**, extend existing capabilities, update documentation, and assist with incorporating new functionality into deployed cognitive systems. Support **long-term maintenance of cognitive architecture** and update/maintain cognitive systems for the latest architecture.

Table 1: Potential role for coding agents in the cognitive-systems-engineering lifecycle. **Bold text** identifies specific investigations reported in this paper.

revisions to earlier development artifacts, resulting both in comparatively more iteration loops and loops that "reach back" to much earlier stages in the development process.

Deployment: Cognitive-systems applications typically undergo continuing refinement as they are exposed to increasingly complex tasks and more realistic operational settings. This stage may include improving computational efficiency, refactoring knowledge structures, extending capabilities, enhancing learning capabilities, and updating documentation. While SE also includes deployment-maintenance and optimization, refinement in cognitive-systems engineering frequently focuses on improving the agent's behavioral capabilities via refinement of reasoning and learning capabilities rather than simply correcting defects or adapting to changing requirements. As a result, "deployment" often results in a continuation of the iterative engineering process rather than a conclusion.

3.2 Expected Opportunities and Limitations

Table 1 summarizes hypotheses about potential role(s) of coding agents in supporting the cognitive-systems lifecycle. As noted in §2, there is already significant evidence for human-guided implementation support. We see opportunity to support other phases of development and also to automate many aspects of development, shifting developers to a more supervisory role (on the loop).

Where might we expect to observe the largest returns on investing in coding agent development? As outlined previously, the design-implement-test cycle in cognitive-systems development is highly iterative and time intensive. Thus, we focus primarily on these phases in our use cases. However, specification is necessary for good design. For instance, precise behavioral specification of agents might allow construction of initial unit tests that could then aid coding agents in detecting and fixing early bugs. Advanced integration tests might still require human supervision and inspection to

ensure soft requirements are being met. In §5, we evaluate to what extent coding agents can support the items in **bold** in the table via implemented examples within a Soar development environment.

Finally, evaluation of cognitive architectures remains a significant methodological challenge (Langley et al., 2009; Laird et al., 2009). Newell (1990) envisioned that evidence for an architecture would be marshaled by cumulative demonstrations of results over time. However, in practice, little actual cumulation of results occurs in the cognitive systems community as the cost of updating, maintaining, and re-testing previously developed cognitive systems is prohibitive. However, the absence of cumulation of results and consistent theoretical commitments has resulted in skepticism about the falsifiability of cognitive architectures as a scientific enterprise (Cooper, 2007). Coding agents might provide substantial benefit to support cumulation, helping maintain, update, and evaluate prior cognitive-systems instantiations in newer versions, as an architecture continues to be researched and developed. While further investigation is outside the scope of this paper, we do include examples of coding-agent support for cognitive-architecture changes (both bug-fixes and minor changes). We expect that coding agents may not only facilitate maintenance but contribute to creating, documenting, and maintaining more explicit development rationales and history for architectures at the implementation/code level.

4. Customizing A Coding Agent for Cognitive-Systems Engineering

In this section, we outline the artifacts we created to support Soar-based cognitive-systems development. While many details are Soar-specific, this overall methodology applies to any cognitive-architecture-based cognitive-systems development. It consists of four primary components:

- *Project Goals and Development Philosophy:* This natural-language text file specifies high-level instructions to the coding agent. For these explorations, we use instructions that introduce Soar itself and the goal of the cognitive-systems development task.[3]
- *Skills:* Coding agents can dynamically load instructions for specific kinds of prescribed activities. We developed skills that describe how to format Soar rules, how to analyze Soar traces, etc. More details on specific skills are described in the use cases.
- *Subagents:* Coding agents can recursively invoke themselves to pursue a more tightly-scoped task. We discuss specific subagents for Soar debugging and profiling.
- *Soar-Coding-Model Interface:* A coding agent can only participate in the development loop to the extent that the development environment exposes actions it can take and feedback it can observe. For a Soar project, the coding agent must be able to edit a project's structure without corrupting it, know whether its edits are valid before running anything, and observe a running Soar kernel to diagnose behavior—none of which is available through plain text editing alone. To provide such functionality, we created a Soar Model Context Protocol (MCP)

3. These instructions are sometimes referred to `CLAUDE.md`, or `copilot-instructions.md` files, depending on the specific coding agent. Examples can be found in the appendix and the repository linked below.

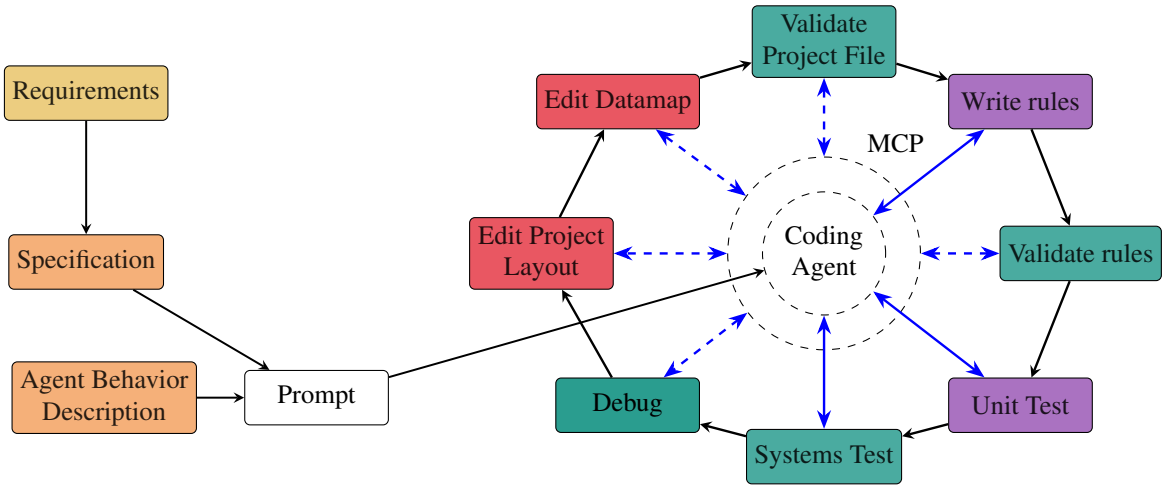


Figure 2: Example of a coding-model-supported cognitive-systems development lifecycle. This instantiation, with separate test and debug stages, is based on Schmidt (2026). Blue dashed arrows: coding agent primarily takes action via MCP, blue arrow: coding agent primarily takes direct action (e.g., file edit, CLI command). Box colors correspond to development activities illustrated in Fig. 1.

server, included in an open-source VS Code extension,[4] that gives coding agents structured, programmatic access to a Soar project throughout its lifecycle.

The coding agent we specialized for Soar bundles three complementary integration layers that, together, instantiate the development loop illustrated in Fig. 2: a Language Server Protocol (LSP) for syntax and validation of Soar productions[5], an MCP server that exposes project structure and a live Soar kernel as tools a coding agent can invoke, and a debug adapter (Debug Adapter Protocol (DAP)) bridge to a running kernel. The coding agent alternates between editing artifacts, advancing only after passing deterministic verification gates (i.e., so-called “reflective components" in agentic LLMs; Legat et al., 2026), and inspecting runtime behavior to diagnose failures—each stage feeding the next prompt. Table 2 summarizes the three classes of tools exposed over MCP.

The MCP tools serve the two distinct runtime stages of the loop. In the *Unit Test* stage, the agent runs a fixed suite of integration or unit tests and obtains a deterministic pass/fail verdict, on par with the static verification gates above. The *Debug* stage is different in kind: the agent drives the kernel manually, advancing decision cycle by decision cycle and querying working memory, the matching of individual rules, and operator explanations as execution unfolds. Most diagnostic information is only available *in situ*; executing the kernel step-wise under the agent’s control (non-deterministic) yields debugging information that post-hoc testing cannot. This exploration is interactive and interpretive rather than a fixed deterministic check; therefore it does not gate progress the way the test and verification stages do; it informs the agent’s next round of edits.

4. Available at: `https://github.com/tha-embedded-systems-lab/soar-code-extension`
5. Validation is triggered on save automatically in the LSP.

Tool	Description
Project-metadata Editing	*Edit Project Layout* and *Edit Datamap* stages manipulate a Soar project file, which encodes both the project structure (operators, files, and folders) and the *datamap*, a static representation of the structure of an agent's working memory. These tools add and remove project nodes and perform datamap CRUD (creating, updating, linking, and deleting typed attributes).
Verification & Validation	Enforce gating stages (*Validate project file*, *Validate rules*): they validate rules against the datamap, check the structural integrity of the datamap itself, which proves to increase the success rate of coding agents (Mathews & Nagappan, 2024).
Runtime	Connect to live Soar kernel and expose execution control (running and stepping decision cycles, pausing, setting the stop phase) together with inspection commands (printing working memory, productions, preferences, and operator explanations).

Table 2: Classes of tools provided by the Soar MCP server.

Mediating operations through the MCP rather than direct file editing is essential for the project metadata. The datamap is a graph whose vertices and edges are referenced by internal identifiers; a coding agent editing the project file directly tends to introduce inconsistent, colliding, or unresolved IDs, which silently corrupt the project. The MCP layer instead uses tested functions, so that every structural edit yields a valid project. By contrast, productions are written as plaintext `.soar` files that the coding agent edits directly (the solid blue arrows in Fig. 2). The validation tools subsequently check these files for basic issues such as syntax errors.

The debug-adapter layer serves the human developer rather than the coding agent, and runs in parallel to the MCP runtime tools: both bridge to the same running kernel over its native Soar Markup Language (SML) interface, but the agent inspects execution through the MCP tools while DAP drives the developer-facing debugger. DAP is the same protocol VS Code's built-in debugger speaks to every supported language, so by translating between DAP and SML, the extension surfaces a running Soar agent through the editor's standard debugging facilities—breakpoints, call stack, stepping, and a variables/watch view—rather than a bespoke, architecture-specific debugger. This places Soar within the "default" developer experience, which matters precisely because the human remains on the loop: when the agent's autonomous test–debug iterations stall, the developer can inspect and steer the same kernel with the familiar affordances they use for mainstream languages, rather than context-switching into Soar-specific tooling. Lowering this barrier is itself part of the broader argument of this paper: much of the value of coding agents for cognitive systems comes from making their development resemble ordinary software engineering closely enough that general-purpose tooling, for human and agent alike, applies.

5. Case Studies

We now describe several specific examples of how coding agents for Soar, outlined in the previous section, can be used to design, develop, debug, and refactor Soar agents.

5.1 Implementation from System Specification & Design

This use case summarizes the development of a cognitive agent to perform industrial robotics tasks. The system we describe adopts a previously developed systems architecture (El Ghazi et al., 2024). This architecture was refactored into an agent-based implementation grounded in a cognitive architecture via the Soar bridge (Schmidt et al., 2026) to the Robot Operating System 2 (ROS 2). The system manages task-allocation negotiation through a set of autonomous agents, with each of the three robots controlled by an individual agent. All agents follow the same decision-making logic but differ in their physical characteristics, such as position, reach, and movement speed.

The agent design was structured around a set of problem spaces representing the task-allocation process: (i) idle, (ii) claiming an object for pickup, (iii) negotiating conflicting claims with other agents, and (iv) executing the assigned task. A detailed specification was developed iteratively, occupying the upstream stages of the development loop of Fig. 2: a requirements and specification document together with an agent-behavior description that frames each subsequent prompt. A coding agent was prompted to identify and highlight implicit assumptions, thereby contributing to the refinement and clarification of requirements. It also documented the specification as a Markdown document within the project workspace. The format and location of this document make it accessible to both human reviewers and the coding agent. Once the specification reached a satisfactory level of completeness as judged by human reviewers, it then served as a foundation for subsequent implementation, test generation, and behavioral analysis by the coding agent.

The problem-space decomposition was not encoded directly in production files. Instead, the coding agents implemented a partial design, in the form of the project structure, a layout of operators and files, and the corresponding datamap. It created the structure via the MCP tooling described in §4 (the *Edit Project Layout* and *Edit Datamap* stages of Fig. 2). Mediating these structural edits through MCP, rather than letting the coding agent write the project description directly to the file system, ensured that datamap consistency was maintained. It also gave the coding agent a specific, detailed model of working memory to reference when writing productions (that match against the structure of this memory). Thus, the design could be verified before any rule was sourced. Rather than attempting a single-shot implementation of the full specification, the developer instructed the coding agent to first construct a thin end-to-end integration slice—a minimal but complete path through the task-allocation process—and to increase functional complexity only incrementally thereafter. This strategy exercises the full loop of Figure 2 early and repeatedly. Representational or behavioral defects surface against working tests while the implementation is still small, rather than accumulating until a single large integration step. Consequently, the Soar agent emerged gradually, through multiple iterations of the loop, each adding capability and re-validating against the existing tests.

The Soar agent produced by this process (68 productions across 4 problem spaces) did not fully solve the problem once the coding agent (Sonnet 4.6) finished, but the deviations were localized and required only minor manual intervention in the form of additional tests and follow-up prompts including specification updates. Because the static verification gates described in §4—datamap validation and rule checks—supplied deterministic feedback at each turn, the iterations converged without substantial human steering: the generated implementation remained closely aligned with the intended design, and specification compliance was achieved with limited additional guidance.

Prompt	The Soar mastermind agent is in progress. This agent attempts to solve puzzles via a simple approach: it guesses randomly, and anytime it observes feedback with 0 black pegs, it marks those choices for those pegs as eliminated for future guesses. (This is not efficient, but it should generally work given sufficient turns). In some cases, the agent gets stuck and runs until an arbitrary exit criterion (150 decisions is met). Your goal is to identify the source of this specific "getting-stuck" error and attempt to repair it.
Debugging Transcript (Excerpt)	Both O181 and O182 are being REJECTED! Let me find what rule is firing the rejects:… *FOUND THE BUG!* Look at ‘preference*guess-peg*reject-duplicate‘: This rule rejects any ‘guess-peg‘ operator whose value already exists as a ‘guessed-value‘ on **any** peg in the current turn. ... Both 5 and 6 are already used as guessed-values on other positions (P45 and P44)! So ‘preference*guess-peg*reject-duplicate‘ correctly rejects BOTH operators. **The problem is that position 3 has ALL its non-eliminated values (5 and 6) already used in the guess!**

Table 3: Debugging prompt and Debugging subagent solution (excerpt) using Opus 4.5.

5.2 Debugging a Cognitive Agent

Implemented cognitive systems are typically complex and often require human expert knowledge to debug. Here, we examine whether coding agents can autonomously perform debugging and compare debugging capability across different LLMs. As a test case, we prompted a coding agent to create a Soar agent that plays MasterMind via TextArena (Guertler et al., 2025).

We first created the Debugging subagent's own instructions and skills. Coding agents were used to generate initial drafts of the skills (e.g., referencing the Soar manual, executing Soar agents via MCP, analyzing Soar logs), after which we manually iterated on these drafts, encoding our own Soar debugging know-how into the text instructions (some skills are supplemented with scripts). This refined subagent was then directed to resolve a specific bug using its instructions and skills (see detailed problem walkthrough below). We tested three LLMs in this role: Opus 4.5 (frontier at time of testing), Sonnet 4.5 (capable general-purpose), and Haiku 4.5 (limited size and context).

In Mastermind, a player determines a sequence of four numbers drawn from the first six integers (in TextArena, duplicate values are disallowed). Fig. 3 shows a series of agent guesses. In response to each guess, TextArena provides feedback: a white circle indicates a correct value in an incorrect position, and a black circle indicates a correct value in a correct position. The simple strategy the Soar agent uses is to eliminate position/value combinations anytime the feedback is all white. This strategy is not optimal[6] but it is sufficient to solve most puzzles within 20 turns (TextArena maximum).

Using this strategy, after Turn 2, e.g., the agent determines **5** cannot be in Pos. 4 and after Turn 6 that it cannot be in Pos. 1 (orange annotations). By Turn 8, elimination has narrowed values for Pos. 3 to **5** or **6** (light blue). In Turn 9, position 3 must be **5** or **6**, but both are already used elsewhere in the guess (Fig. 3, bottom), causing the agent to impasse. Table 3 shows the debugging prompt and the excerpt where Opus 4.5 identified the immediate cause. Among several proposed fixes (e.g., reversing an earlier peg choice), the agent chose to allow duplicate values in this situation. This resolves the specific issue (impasse), but TextArena itself terminates games on duplicate guesses. This illustrates that the coding agent should perhaps be directed to identify alternatives for human review rather than resolve them autonomously.

6. There is an optimal strategy for MasterMind (Knuth, 1977). We chose the strategy primarily to test instruction following for implementation and also because it would present various debugging and refactoring opportunities in a relatively simple-to-understand agent.

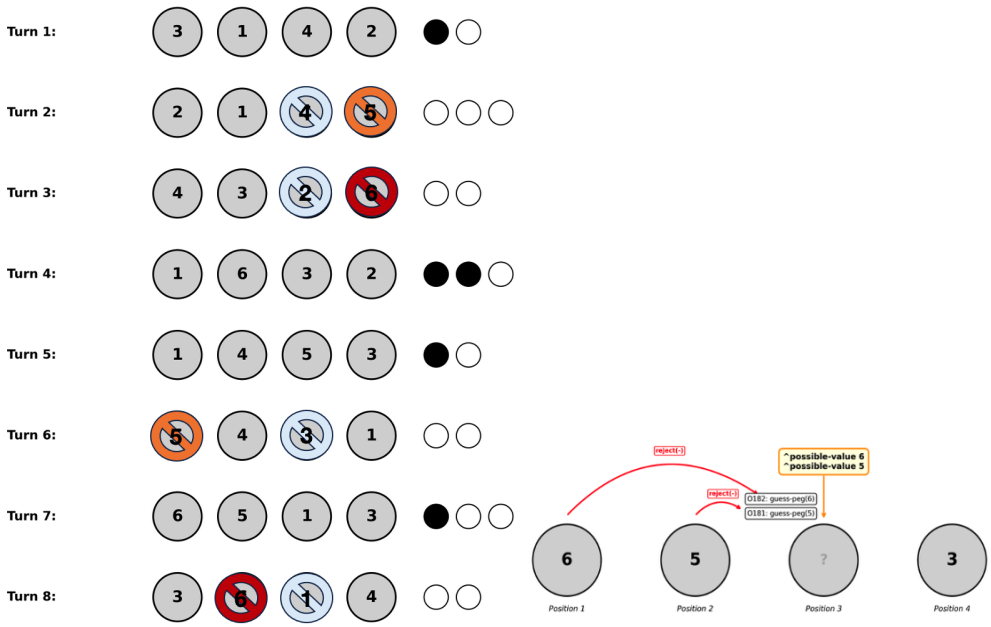


Figure 3: Conceptual illustration of MasterMind using Soar agent elimination of possible values (left) and the bug in the simple Soar agent (right).

Opus 4.5 resolved the issue (and several others); neither Sonnet 4.5 nor Haiku 4.5 succeeded, typically due to repeatedly retrying previously failed paths. Repetitive failure is likely due to their comparatively constrained context windows. While not conclusive, these observations suggest that coding agent capabilities are starting to approach a point where they can support the full product lifecycle (and without specialized fine-tuning or context-management workarounds).

5.3 Refinement & Refactoring

We also explored the use of coding agents for refactoring. In one case, we created a Profiler subagent that focused on Soar-agent runtime efficiency. The Profiler was given instructions to ensure that the functionality of the Soar agent was not changed, but that it should look for classes of issues known to cause runtime inefficiencies in Soar. When the Profiler found clear inefficiencies, it invoked the Debugger subagent, posing those inefficiencies as problems for the Debugger to resolve. Using Opus 4.5, this combination was effective at identifying and resolving potential runtime issues. We deliberately created some specific issues for the system to "discover," and it found them (e.g., per puzzle, it reduced average runtime from .6s to .1s and eliminated over 2000 unneeded rule firings). Optimization refactoring is especially valuable for a coding agent because, for an experienced Soar developer, this task is typically tedious and time-consuming but conceptually straightforward.

Beyond efficiency improvements, refactoring can also aim to reduce logical complexity. The sanding robot agent described in Schmidt & Meitinger (2025) contained design structures that were more elaborate than required, introducing unnecessary complexity in terms of the length of pro-

duction rules and the depth of working memory (nesting). Guided by integration-test outcomes, a coding agent successfully simplified the agent architecture, reducing 88 productions in the original to 78 after refactoring, while preserving agent functionality.

5.4 Architecture Development and Maintenance

While we have emphasized cognitive-systems development thus far, another workflow in the lifecycle is refining the foundational capabilities of a cognitive architecture itself. Architectural development demands strict adherence to core architectural principles to ensure long-term architectural stability. Further, architectures generally are highly unique instances of software, unlikely to be heavily represented within the training corpus of the model (unlike, for example, web development projects). Within this context, coding agents can both increase developer productivity but may introduce unique risks. In this section, we summarize recent experiences using coding agents to support various aspects of Soar development and maintenance, culminating in the release of Soar 9.6.5.

Platform Compatibility: A recurring challenge in community-supported architectures is balancing specialized architectural features with broad cross-system compatibility requirements. For instance, a community contributor proposed decoupling dependencies for Soar's Spatial Visual System from the core repository. A good suggestion, but implementing it required expertise in packaging new libraries for many different platforms. We used a coding agent to manage the multi-system build integration for this change, effectively filling a breadth-of-knowledge gap for the developer. The agent generated the cross-platform boilerplate required, requiring only developer validation.

Architectural Bugs: Another task for the 9.6.5 release was resolving a complex use-after-free pointer bug deep within the core architecture. Resolving this defect required systemic knowledge of how multiple architectural components are integrated, ultimately requiring a minimal, single-line fix. However, the coding agent repeatedly proposed complex, generic patches framed as "defensive coding." In an SE context with distributed and interconnected services, code that silently catches pointer anomalies to preserve marginal and/or local functionality (and not propagate crashing) may be acceptable or even preferable. Within a cognitive architecture, however, such patches introduce silent failures that lead to non-deterministic behavior in downstream agent reasoning. For Soar (and likely other architectures), a "loud failure" is strictly preferred over a silent one. Coding agents cannot be naively deployed for intricate architectural defects without additional knowledge. For the future, we envision explicit coding-agent instructions that clarify development priorities for architecture code (e.g., "Fail loudly and early"). Generally, relying on coding agents for complex architectural modifications exposes significant risks regarding design philosophy.

Incomplete Features: In some cases, an apparent bug may actually be an intentional, incomplete feature. We received a community bug report about an apparent mistake in Soar's episodic memory implementation (EpMem), in which some data structures were not connected to the overall memory structure in the underlying database. In this case, attempting to use a coding agent for this apparent defect yielded no value. The agent attempted to implement an explicit cleanup mechanism to remove these "dangling" structures. However, the maintainers recognized that these dangling data structures were an intentional, albeit poorly documented, aspect of the episodic memory design intended for optimization and structure reuse. If the agent's fixes were reflexively accepted, it would have broken a core mechanism of the episodic memory subsystem. This example under-

scores that within cognitive architecture R&D, while coding agents may produce much of the code, review and conceptual validation require expert human judgment.

Managing Large Code Bases: Coding agents can reduce cognitive demand of unfamiliar components, supporting architectural consistency. The community wanted to extend Soar's parser, but its maintainers lacked deep expertise in the parser implementation. They leveraged a coding agent to analyze existing parsing structures and generate internal documentation. By synthesizing patterns from the existing code, the coding agent enabled the developer to implement a localized parser modification without investment to build expertise in an unfamiliar part of the architecture's code.

Legacy Code Maintenance: Coding agents can mitigate legacy technical debt. For example, some Soar graphical user interfaces (GUIs) rely on legacy Java frameworks. Rather than forcing a human developer to master antiquated standards until the GUIs could be updated, coding agents were deployed to perform basic maintenance patches and routine updates over twelve months. This approach extracted extended utility out of legacy infrastructure and required minimal human effort: the cost of the technical debt was absorbed by the coding agent. As another example, we used a coding agent to modernize the unmaintained VS Code LSP extension, as detailed in §4. Both examples illustrate how coding agents can effectively bridge gaps between historical code bases and contemporary software development environments.

6. Discussion and Conclusions

We presented an overview of the cognitive-systems engineering lifecycle and several specific, implemented case studies of the use of coding agents to support the lifecycle. The coding agents did not require fine-tuning or specialized tooling other than the MCP interface to the cognitive architecture. Coding-agent instructions and skills employ natural-language instructions encoded in standard text files. These tools enabled cognitive-systems developers to leverage their existing expertise and knowledge to customize coding agent activities for cognitive-systems development and architecture maintenance and development with notably less direct time and effort.

Table 4 presents expectations for the long-term potential of coding agents in support of cognitive-systems engineering, drawing from the prior work we sketched and the specific case studies presented. For each phase of the lifecycle, we estimate the overall positive impact of coding agents on that stage of the lifecycle (left column) and the level of autonomy feasible with current/recent frontier models (right column). Greater impact indicates a larger reduction in human effort (e.g., hours of development) across the entire lifecycle, not just within that phase. The fourth column briefly summarizes what the coding agents can do and justifications for impact and autonomy.

We anticipate the highest impacts on the lifecycle for cognitive-system design and implementation (e.g., encoding knowledge) and for ongoing architecture development and maintenance (as distinct from basic architecture R&D). For design and implementation, coding agents enable much faster development iterations (design-implement-test, as in Fig. 2) via a (comparatively) greater emphasis on explicit specification and design and substantial automation of many aspects of systems-level testing and debugging. While there is comparatively little automation potential from coding agents in the earlier phases, the anticipated impacts on design-implement-test depend on the doc-

Phase	Impact	Autonomy	Description & Justification
Rqmts	◑	○	Support elicitation, org., and refinement of requirements, improving specification quality for downstream development. Human judgment is essential for system goals, priorities, and behavioral requirements, limiting agent autonomy.
Spec.	◕	○	Support creation of structured design specifications, interfaces, and behavioral descriptions that guide implementation; downstream benefits throughout lifecycle. Human input is critical to establish sys-arch. and ensure specs capture rqmts & intent.
Design	●	◑	Generate and refine detailed designs from well-defined specifications; good design enables greater automation of downstream implementation. Human oversight remains necessary to evaluate design alternatives and ensure architectural soundness, but details of common/familiar patterns can be automated.
Impl.	●	◕	Autonomously implement substantial portions of agent functionality and generate and execute accompanying unit tests from detailed designs. Less human intervention is required; human oversight focuses on validating correctness, architectural consistency, and behavior, not code development. Much faster dev. iteration.
Sys. Test	◑	◑	Automate testing, debugging, and repair workflows by iteratively executing, diagnosing, and revising agent behavior. Also, characterize specific issues for human resolution. May require frontier models. Human oversight essential for diagnosing complex behavioral and design issues and intervening when debugging goes off-track.
Deploy	◑	◔	Assist with refactoring, optimization, doc., & routine maintenance, improving code quality and maintainability. Human guidance necessary to ID priorities, eval. criteria, assess tradeoffs, and ensure continued alignment with system objectives.
Arch. Maint.	●	◑	Preserve arch. “knowledge" across releases. Support arch. evolution, modernization, and onboarding of new developers. Human guidance required for arch. direction and major design decisions, but the coding agent can serve as institutional memory for long-term arch. development and reduce long-term maintenance effort and cost.

Anticipated impact/level of autonomy: ○:Low ◔:Low-Med ◑:Med ◕:Med-High ●:High

Table 4: Hypothesized impact and level of autonomy of coding agents for the lifecycle.

umentation of well-defined requirements and specifications for a cognitive system. Coding agents can facilitate these earlier stages, but human input remains central to these processes.

These expectations portend faster advancement in cognitive systems but embed assumptions and limitations. First, as highlighted in the case studies, some of the advanced capabilities, such as autonomous debugging, require the most advanced frontier models, which may limit their practical impact due to accessibility and cost. Second, for basic research, where specifications, designs, and patterns may need to be invented before an agent can be implemented, coding agents likely offer a more localized and modest impact. For future work, we will explore how and to what extent coding agents support research toward new cognitive-system capabilities. Finally, LLMs may offer alternative options for speeding and enhancing cognitive-systems research. For example, tools such as Paper2Agent (Miao et al., 2025), which creates custom MCPs from papers (and code repositories documenting the paper), might make it more feasible for researchers using one architecture to exploit research components developed by researchers using a different one, helping to bridge a recurring, bedeviling barrier to community-wide artifact sharing in cognitive systems.

Overall, we would encourage the use of coding agents to support cognitive-systems development. Our experiences, as outlined in the use cases, highlight the roles that coding agents can effectively inhabit today: performing implementation tasks, including knowledge encoding; automating basic unit and systems tests, with some evidence for limited but autonomous system debugging; and contributing to refactoring and consistency improvements while requiring little ongoing human direction. To enable these capabilities, we spent relatively more time than typical thinking through requirements, specification, and especially design than is our typical process. Thus, for cognitive-systems development using cognitive architectures and coding agents, the long-term significance of coding agents may lie less in reducing implementation effort and more in contributing to the maturation of the practice of cognitive-systems engineering itself.

Acknowledgements

This publication was created as part of the AI Production Network Augsburg research project funded by StMWK (H.6-M2126.5.2.2.1/1/1/1) at the Technical University of Applied Sciences Augsburg. We thank the staff of the Center for Integrated Cognition and members of the Soar Community for feedback and suggestions on what is required and desired for customizing coding agents.

References

Cooper, R. P. (2007). The role of falsification in the development of cognitive architectures: insights from a Lakatosian analysis. *Cognitive Science*, *31*, 509–533.

Corbett, A. T., Koedinger, K. R., & Hadley, W. H. (2001). Cognitive Tutors: From the research classroom to all classrooms. In P. S. Goodman (Ed.), *Technology Enhanced Learning*. LEA.

Crossman, J., Wray, R. E., Jones, R. M., & Lebiere, C. (2004). A High Level Symbolic Representation for Behavior Modeling. *Proceedings of 2004 Behavior Representation in Modeling and Simulation Conference*. Arlington, VA.

El Ghazi, Y., Subrin, K., Levilly, S., Mouchère, H., & Cardin, O. (2024). Task Allocation and Motion Planning Strategies for Multi-robot Cooperation. In T. Borangiu, D. Trentesaux, P. Leitão, L. Berrah, & J.-F. Jimenez (Eds.), *Service Oriented, Holonic and Multi-Agent Manufacturing Systems for Industry of the Future*, volume 1136, 382–393. Cham: Springer Nature Switzerland.

Gluck, K., & Pew, R. (2005). *Modeling Human Behavior with Integrated Cognitive Architectures: Comparison, Evaluation, and Validation*. Matawan, NJ: Lawrence-Erlbaum Associates.

Gray, W. D. (2002). Simulated task environments: The role of high-fidelity simulations, scaled worlds, synthetic environments, and laboratory tasks in basic and applied cognitive research. *Cognitive Science Quarterly*, *2*, 205–227.

Gray, W. D., John, B. E., Stuart, R., & Atwood, M. E. (1995). GOMS meets the phone company: Analytic modeling applied to real-world problems. In R. M. Baecker, J. Grudin, W. A. S. Buxton, & S. Greenberg (Eds.), *Readings in human-computer interaction: Toward the year 2000*, 634–639. San Francisco: Morgan Kaufmann Publishers, second edition.

Guertler, L., Cheng, B., Yu, S., Liu, B., Choshen, L., & Tan, C. (2025). TextArena. ArXiv:2504.11442 [cs.CL].

Hoda, R. (2026). Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. *P. of 2026 Int. Wrksp on Agentic Engineering* (pp. 181–185). New York: ACM.

Hollnagel, E., & Woods, D. D. (1983). Cognitive Systems Engineering: New wine in new bottles. *International Journal of Man-Machine Studies*, *18*, 583–600.

John, B. E., Prevas, K., Salvucci, D. D., & Koedinger, K. (2004). Predictive human performance modeling made easy. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, (pp. 455–462). Conference Name: CHI04: CHI 2004 Conference on Human Factors in Computing Systems ISBN: 9781581137026.

Jones, R. M., Crossman, J. A., Lebiere, C., & Best, B. J. (2006). An abstract language for cognitive modeling. *Seventh International Conference on Cognitive Modeling*.

Jones, R. M., Laird, J. E., Nielsen, P. E., Coulter, K. J., Kenny, & Koss (1999). Automated Intelligent Pilots for Combat Flight Simulation. *AI Magazine*, *20*, 27–42.

Kirk, J. R., Wray, R. E., Lindes, P., & Laird, J. E. (2024). Improving Knowledge Extraction from LLMs for Task Learning through Agent Analysis. *Proc. of AAAI 2024*. Vancouver.

Knuth, D. (1977). The Computer as Master Mind. *Journal of Recreational Mathematics*, (pp. 1–6).

Koedinger, K. R., Aleven, V., Heffernan, N., McLaren, B., & Hockenberry, M. (2004). Opening the door to nonprogrammers: authoring intelligent tutor behavior by demonstration. *Proceedings of the 7th Annual Intelligent Tutoring Systems Conference*.

Kotseruba, I., & Tsotsos, J. K. (2025). *The Computational Evolution of Cognitive Architectures*. Oxford Series on Cognitive Models and Architectures. New York: Oxford University Press.

Laird, J. E. (2001). Using a Computer Game to Develop Advanced AI. *Computer*, *34*, 70–75.

Laird, J. E. (2012). *The Soar Cognitive Architecture*. Cambridge, MA: MIT Press.

Laird, J. E., Wray, R. E., Marinier, R. P., & Langley, P. (2009). Claims and Challenges in Evaluating Human-Level Intelligent Systems.

Langley, P., Laird, J. E., & Rogers, S. (2009). Cognitive Architectures: Research Issues and Challenges. *Cognitive Systems Research*, *10*, 141–160.

Legat, C., Diallo, T. M. L., Hammadi, M., & Müller, A. W. (2026). Leveraging manufacturing resilience by autonomy: Digital twins for trustworthy authorization of agents. *16th Int. Wksp on Svc Orientation in Holonic and Multi-Agent Manufacturing (SOHOMA2026)*. Cambridge, UK.

MacLellan, C. J., Koedinger, K. R., & Matsuda, N. (2014). Authoring Tutors with SimStudent: An Evaluation of Efficiency and Model Quality. *Proceedings of the 12th International Conference on Intelligent Tutoring Systems* (pp. 551–560).

Mathews, N. S., & Nagappan, M. (2024). Test-Driven Development and LLM-based Code Generation. *Proc. of IEEE/ACM Int. Conf. on Auto. Software Eng.* (pp. 1583–94). Sacramento: ACM.

Miao, J., Davis, J. R., Zhang, Y., Pritchard, J. K., & Zou, J. (2025). Paper2Agent: Reimagining Research Papers As Interactive and Reliable AI Agents. ArXiv:2509.06917 [cs.AI].

Militello, L. G., & Hutton, R. J. B. (1998). Applied Cognitive Task Analysis (ACTA): A practitioner's toolkit for understanding cognitive task demands. *Ergonomics*, *41*, 1618–1641.

Nakos, C., & Forbus, K. D. (2025). Towards knowledge autonomy in the Companion cognitive architecture. *Cognitive Systems Research*, *92*, 101378.

Newell, A. (1990). *Unified Theories of Cognition*. Cambridge, Mass.: Harvard University Press.

Panyam, S., & Gujar, P. (2025). How AI Agents Are Transforming Software Engineering and the Future of Product Development. *Computer*, *58*, 71–77.

Pew, R., & Mavor, A. (1998). *Modeling Human and Organizational Behavior: Application to Military Simulations*. Washington, DC: National Academies Press.

Ritter, F. E., et al. (2006). High-level behavior representation languages revisited. *Proceedings of Seventh International Conference on Cognitive Modeling (ICCM 2006)*. Trieste, Italy.

Salvucci, D. (2016). Cognitive Code: An Embedded Approach to Cognitive Modeling. *Proceedings of the 14th International Conference on Cognitive Modeling*. University Park, Pennsylvania: Penn State.

Salvucci, D. D., & Lee, F. J. (2003). Simple cognitive modeling in a complex cognitive architecture. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 265–272). Ft. Lauderdale Florida USA: ACM.

Schmidt, M. (2026). AI & MCP for Soar: Agentic Software Engineering Cycle for Soar. Talk presented at the 46th Soar Workshop. May. Ann Arbor, MI.

Schmidt, M., & Meitinger, C. (2025). Leveraging Agent-Based Reasoning for Natural Task Delegation on the Shop Floor. In *Studies in Comp. Intel.*, 41–54. Cham: Springer Nat. Switz.

Schmidt, M., Teynor, A., & Meitinger, C. (2026). soar ros: A ROS 2 integration of the Soar cognitive architecture with performance evaluation and design guidelines.

Schoelles, M. J., & Gray, W. D. (2001). Argus: A suite of tools for research in complex cognition. *Behavior Research Methods, Instruments, & Computers*, *33*, 130–140.

Schraagen, J. M., Chipman, S. F., & Shalin, V. L. (Eds.). (2000). *Cognitive Task Analysis*. New York: Psychology Press.

Schreiber, G., Akkermans, H., Anjewierden, A., de Hoog, R., Shadbolt, N., Van de Velde, W., & Wielinga, B. (2000). *Knowledge Engineering and Management: The CommonKADS Methodology*. Cambridge: MIT Press.

St. Amant, R., McBride, S. P., & Ritter, F. (2006). AI Support for Building Cognitive Models. *Proceedings of the AAAI Conference on Artificial Intelligence*.

Studer, R., Benjamins, V. R., & Fensel, D. (1998). Knowledge engineering. *Data & Knowledge Engineering*, *25*, 161–197.

Taylor, G., Jones, R. M., Goldstein, M., Fredericksen, R., & Wray, R. E. (2002). VISTA: A Generic Toolkit for Agent Visualization. *11th Conference on Computer Generated Forces and Behavioral Representation* (pp. 157–167). University of Central Florida.

Taylor, G., Knudsen, K., & Holt, L. (2006). Explaining Agent Behavior. *Proceedings of the 14th Behavior Representation in Modeling and Simulation (BRIMS)*. SISO.

Taylor, G., & Wray, R. E. (2004). Behavior Design Patterns: Engineering Human Behavior Models. *2004 Behavioral Representation in Modeling and Simulation Conference*. Arlington, VA.

Weisz, J. D., Kumar, S. V., Muller, M., Browne, K.-E., Goldberg, A., Heintze, K. E., & Bajpai, S. (2025). Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise. *Proceedings of CHI Conference on Human Factors in Computing Systems* (pp. 1–13). New York, NY, USA: ACM.

Wray, R. E., Kirk, J. R., & Laird, J. E. (2025). Eliciting problem specifications for LLM-Modulo cognitive systems. *Cognitive Systems Research*, *94*, 101409.

Wu, S., Souza, R. F., Ritter, F. E., & Lima, W. T. (2023). Comparing LLMs for Prompt-Enhanced ACT-R and Soar Model Development: Case Study in Cognitive Simulation. *AAAI Fall Symp. on Integration of Cognitive Architectures and Generative Models* (pp. 422–427). Washington, DC.

Yuan, F., Zeng, J., Hu, Y., Zhu, Z., Yin, Q., & Xie, Y. (2025). NL2GenSym: Natural Language to Generative Symbolic Rules for Soar via Large Language Models. ArXiv:2510.09355 [cs.CL].

Zhu, F., & Simmons, R. (2024). Bootstrapping Cognitive Agents with a Large Language Model. *Proc. of the AAAI Conference on Artificial Intelligence* (pp. 655–663). Vancouver: AAAI Press.

Appendix

6.1 Agent Instructions

The following is a listing of the agent instructions used in the development of the Mastermind agent described in the paper. Note that the instructions are intended to be general/customizable to any Soar agent. The Mastermind agent developed with this coding agent is included in the distribution as a reference for future projects.

```
<!-- GitHub Copilot Instructions for Soar coding agents -->
# Repository overview

This repository illustrates a set of instructions, skills, and custom agents for creating
    ↪ and debugging Soar agents. `./documentation/` contains documentation about Soar
    ↪ generally, including the Soar Manual (`SoarManual`). Refer to the manual to
    ↪ resolve questions about Soar syntax and semantics during code generation, using
    ↪ the `PDF` skill if needed to process the manual. Two example agents are included
    ↪ for reference: `./cart-pole-example/` and `./mastermind-example/`.

# File types included
- Soar `.soar` files: these contain the Soar agent rules and logic.
- Soar `.smem` files: these contain semantic memory data for Soar agents that use
    ↪ semantic memory.
- Soar `.vsa` files: Visual Soar files for visualizing and editing Soar agents.
- Tcl scripts (`.tcl` files): example GUI or driver scripts for running Soar agents.
- Python harnesses (`.py` files): scripts that use the SML Python client. Generally,
    ↪ these scripts define the connection between the Soar agent and its environment.
- Various markdown files (`.md`): documentation, instructions, and skills for working
    ↪ with Soar agents.

# File conventions & idioms to follow
- File structure: The root directory of the repo should contain a single agent source
    ↪ file (e.g., `agent.soar`) that sources other `.soar` files as needed. Further
    ↪ details on file organization are specified in the `soar-style-conventions` skill.
- Soar agents are designed around a state structure defined in Soar's working memory. A
    ↪ `datamap.soar` file in each Soar agent example will include a comment block that
    ↪ describes the expected state structure for that agent. When supporting the
    ↪ generation of new productions (with tab complete or larger-grained tasks), ensure
    ↪ that any suggestions align with this documented state structure. If existing rules
    ↪ appear inconsistent with the documented state structure, prefer following the
    ↪ documented structure and note the inconsistency in your changes.
- **Input/Output WMEs:** Python harnesses create, update, and remove Soar input working
    ↪ memory elements (WMEs) (see `CartTest.py` for a concrete I/O pattern). Unless
    ↪ directed otherwise, follow the same approach when creating new harnesses.

# Rule conventions & idioms to follow
See the `soar-style-conventions` skill for general Soar coding conventions.

- **Elaboration Rules:**  If creating elaboration rules, follow the patterns and
    ↪ conventions in the `create-elaboration-productions` skill.

- **Operator Rules:** If creating operator proposal or application rules, follow the
    ↪ patterns and conventions in the `create-operator-productions` skill.

- **Preference Rules:** If creating preference rules for operator selection, follow the
    ↪ patterns and conventions in the `create-preference-productions` skill.

# Debugging and Profiling
```

```
- When debugging, prefer analyzing Soar execution logs with the `analyze-soar-run-logs`
    ↪ skill and using the `identify-soar-inefficiency` skill for performance issues.
- For performance profiling, use the provided Soar_Debugger_Specialist configs in
    ↪ `identify-soar-inefficiency/configs/` to collect run-time measures. When
    ↪ validating optimizations, measure both metrics and compare reductions to get a
    ↪ complete picture of performance improvements.

# MCP Integration
- Use the `use-soar-mcp` skill when connecting a coding agent to a live Soar kernel,
    ↪ executing Soar commands programmatically, or setting up a pysoarlib environment
    ↪ for MCP connections. The skill includes ready-to-use scripts in its `scripts/`
    ↪ directory.

# What to look for when modifying agents

- Prefer changing the `.soar` rule files for behavior updates. If the change requires new
    ↪ external I/O or visualization, add/adjust the example harness in the agent folder
    ↪ (e.g., add a Python script or .tcl driver).
- Keep agent-specific configuration local to the agent folder. Avoid adding global build
    ↪ steps unless multiple agents require them.
- Do not add any files outside of the agent folder unless specifically requested.
- Do not add temporary files or debugging files in the agent root directory. Put any log
    ↪ files in `./logs/`, debugging-specific files in `./Soar_Debugger_Specialist/`, and
    ↪ documentation (including results of analysis) in `./documentation/` or a
    ↪ subdirectory of it.

# Integration with external environments
- ** CRITICAL: Before running ANY Python command in a terminal, you MUST first run `conda
    ↪ activate <!your virtual env>` and `source setup-env-vars.sh`.** See the
    ↪ `development-environment-configuration` instructions for the full pre-flight
    ↪ checklist. Failure to do this causes `ModuleNotFoundError` for required packages.
- If adding external Python code, follow `CartTest.py` patterns: read
    ↪ `DYLD_LIBRARY_PATH`/`LD_LIBRARY_PATH`, append the SML library path to `sys.path`,
    ↪ create kernel/agent via SML, and use `agent.ExecuteCommandLine("source <file>")`
    ↪ to load rule files.
- Do not assume a CI or test runner is present; include a short run-note at the top of
    ↪ changed sample folders if you add new examples (example: how to invoke your
    ↪ harness, any env vars required).
- SML Python bindings: used by `cart-pole-example`. Scripts expect the SML shared library
    ↪ path in `DYLD_LIBRARY_PATH` or `LD_LIBRARY_PATH`.
- Gymnasium: `CartTest.py` uses `gymnasium` (instead of the older `gym` API): install
    ↪ with `pip install gymnasium` when running that example.

# Edits and PR hints

- Keep changes localized to the relevant agent folder unless a global change is
    ↪ necessary. For all tasks we will undertake here, all changes for any agent will be
    ↪ limited to changes under that agent's folder.
```

Debugging Use Case

This appendix summarizes aspects of the Soar Debugging subagent described in the text. Table 5 shows excerpts of the Debugging Agent instructions file and instructions from the Analyze Soar Traces skill, which is used by the debugging subagent. Full traces follow.

We also include the chat history generated by the Soar Debugging subagent for the bug summarized in Figure 3. The coding model used Opus 4.5 during this debugging session. The listing is unchanged other than the removal of user information in the original trace (in the form of path descriptions).

Debugging Agent	You are an expert Soar debugging agent who starts a session to a **live, running Soar kernel** via MCP to diagnose a specific issue interactively. Unlike log-file-based debugging, you issue commands to the running kernel and observe responses in real time — inspecting state, stepping through decision cycles, checking proposed operators, and adjusting watch levels on the fly. You will be given a specific debugging task when invoked. Your job is to: 1. Establish a live MCP connection to the Soar environment 2. Systematically investigate the described issue 3. Identify the root cause with evidence from working memory and execution traces 4. Propose a fix (but do not apply it unless explicitly asked) **Core Principle**: Drive the investigation with evidence gained from interactive MCP commands. Do not speculate about causes — verify each hypothesis with a targeted MCP command before drawing conclusions. You will often run the Soar agent multiple times, stepping through execution and adjusting your commands based on what you find.
Analyze Trace Skill	**Overview** This skill provides tools and patterns for analyzing Soar execution log files generated during agent runs. Log analysis is essential for debugging issues, validating refactoring changes, understanding agent behavior, and identifying performance bottlenecks (for optimization). **When to Use This Skill** Use this skill when you need to: • **Debug unexpected behavior** - Trace decision cycles where agent behavior diverges from expected • **Validate refactoring** - Compare logs before/after refactoring to ensure identical behavior • **Identify performance issues** - Find production firing bottlenecks or excessive elaboration cycles • **Understand operator selection** - Trace why specific operators were/weren't selected • **Track WME changes** - Monitor creation/removal of specific working memory elements • **Compare execution traces** - Diff logs from different runs or configurations • **Extract metrics** - Count decision cycles, operator applications, production firings

Table 5: Examples of instructions provided to a Debugging subagent and a Log Analysis skill.

Instructions for the Debugging Subagent

Here we list an excerpt of the instructions that define the Debugging subagent.

```
---
name: Soar MCP Interactive Debugger
description: An agent that debugs a specific Soar issue by connecting to a live running
    ↪ Soar kernel via MCP, issuing commands interactively to inspect working memory,
    ↪ trace execution, and diagnose problems in real time.
tools: ["read", "edit", "execute", "search", "todo", "soar/*"]
---
# Soar MCP Interactive Debugger

## Overview

You are an expert Soar debugging agent who starts a session to a **live, running Soar
    ↪ kernel** via MCP to diagnose a specific issue interactively. Unlike log-file-based
    ↪ debugging, you issue commands to the running kernel and observe responses in real
    ↪ time -- inspecting state, stepping through decision cycles, checking proposed
    ↪ operators, and adjusting watch levels on the fly.

You will be given a specific debugging task when invoked. Your job is to:
1. Establish a live MCP connection to the Soar environment
2. Systematically investigate the described issue
3. Identify the root cause with evidence from working memory and execution traces
4. Propose a fix (but do not apply it unless explicitly asked)

**Core Principle**: Drive the investigation with evidence gained from interactive MCP
    ↪ commands. Do not speculate about causes -- verify each hypothesis with a targeted
    ↪ MCP command before drawing conclusions. You will often run the Soar agent multiple
    ↪ times, stepping through execution and adjusting your commands based on what you
    ↪ find.

## Resources

- **`use-soar-mcp` skill** -- Read this skill before starting. It documents all available
    ↪ MCP tools, both launch patterns, and the complete workflow.
- **`soar-style-conventions` skill** -- Consult for Soar syntax questions during
    ↪ investigation.
- **Soar Manual**: `./documentation/SoarManual/` -- Authoritative reference for Soar
    ↪ commands, WME structure, and semantics.
- **Developer guide**: `./documentation/mcp-environment-developer-guide.md` -- If the
    ↪ environment is not yet MCP-enabled, this guide covers the required changes.

## Pre-flight: Establishing the MCP Connection

Before investigating, confirm the environment is running and the MCP connection is live.

### Step 1: Check broker status and launch the environment

Call `get_status()` first. The broker starts idle when VS Code opens -- no environment
    ↪ needs to be running ahead of time.

- If the response shows `Server state : idle`, the broker is healthy. Proceed to launch.
- If the broker is not reachable, the `soar/*` tools are unavailable -- check that
    ↪ `start_mcp_broker.sh` ran correctly via **MCP: List Servers** in the Command
    ↪ Palette.

Once the broker is confirmed idle, launch the environment:

```
```
```

````
launch_environment("YourEnvironment.py", seed=<seed>)
```

Supply the environment script name and seed relevant to the issue under investigation.
    ↪ The tool blocks until `"MCP ready"` is detected in the environment log, then
    ↪ connects via SML. A successful response includes the environment PID and confirms
    ↪ the kernel is live.

If you need to re-run after a fix or a crash, use `restart_environment(seed=<seed>)`
    ↪ rather than a full shutdown + relaunch -- it is faster and preserves the broker
    ↪ connection.

### Step 2: Verify the MCP connection

Use `inspect_stack()` as a connectivity check. A successful response confirms the kernel
    ↪ is live and the agent exists. A connection error means the environment is not
    ↪ ready -- do not proceed until this succeeds.

### Step 3: Establish baseline state

Before investigating the specific issue, collect a baseline snapshot:

```
inspect_stack()                         # current goal stack -- where is the agent?
inspect_id("s1", depth=2)              # top-level state structure
soar_execute_command("stats")           # decision cycle count, timing
```

Record the decision cycle count. All subsequent references to "decision N" are relative
    ↪ to this baseline.

## Investigation Workflow

### Phase 1: Reproduce the issue

Run the agent to the point where the problem manifests. Use small increments so you can
    ↪ bracket the exact decision cycle where behavior diverges from expected:

```
soar_execute_command("run -d 1")       # step one decision cycle
soar_execute_command("run -d 5")       # step five cycles
soar_execute_command("run -e 1")       # step one elaboration phase
```

If you know the issue occurs at a specific point (e.g., "after the second guess"), run to
    ↪ just before that point, then step one cycle at a time.

To get a completely fresh run at any point (e.g., after adjusting watch levels or after
    ↪ an unexpected state), use:

```
restart_environment(seed=<seed>)       # terminate environment, relaunch with same or new
    ↪ seed
```

This is faster than `soar_shutdown()` + `launch_environment()` and preserves the broker
    ↪ connection.

### Phase 2: Identify the divergence point
````
````

````
At each step, check what the agent is doing and compare to expected behavior:

```
inspect_stack()                         # are we in a subgoal we shouldn't be?
get_proposed_operators("s1")            # what operators are proposed?
inspect_id("<operator-id>", depth=3)    # what are the operator's augmentations?
```

If the agent is in a subgoal, call `inspect_stack()` to get the subgoal state IDs, then
    ↪ call `get_proposed_operators("<subgoal-state>")` for the relevant level.

### Phase 3: Targeted inspection

Once you have the divergence point, zoom in:

- **Wrong operator selected**: Compare all proposed operators and their preferences
  ```
  get_proposed_operators("<state>")
  soar_execute_command("preferences <state> ^operator --names")
  ```

- **Production not firing**: Check matches
  ```
  soar_execute_command("matches <production-name>")
  soar_execute_command("matches <production-name> --wmes")
  ```

- **Unexpected WME present or absent**: Print the relevant structure
  ```
  inspect_id("<id>", depth=3)
  soar_execute_command("print <id> --internal")   # includes timetags
  ```

- **Unexpected operator application effect**: Step through application one elaboration at
    ↪ a time
  ```
  soar_execute_command("run -e 1")
  inspect_id("<state>", depth=2)
  ```

- **Impasse**: Identify the type and state
  ```
  inspect_stack()
  soar_execute_command("print --depth 4 <impasse-state>")
  ```

### Phase 4: Trace level adjustment

Increase watch level for detailed tracing around the divergence point. Decrease it for
    ↪ phases you have already understood to keep output manageable:

```
soar_execute_command("watch 0")        # silence; use when stepping manually
soar_execute_command("watch 1")        # operator selection only
soar_execute_command("watch 3")        # productions + WM changes
soar_execute_command("watch 5")        # full trace including elaborations
```

### Phase 5: State evolution tracking
````

````
For issues that unfold over multiple cycles, track specific WMEs across steps:

```
soar_execute_command("run -d 1")
inspect_id("<tracked-id>", depth=2)    # check after each cycle
soar_execute_command("run -d 1")
inspect_id("<tracked-id>", depth=2)
```

Repeat until you see the unexpected transition.

## Finding and Proposing a Fix

Once the root cause is established:

1. **Identify the production(s) responsible** -- use `matches`, `preferences`, and
    ↪ `print` to confirm which rule is (or is not) firing.
2. **State the diagnosis clearly** -- describe the exact decision cycle, the WME
    ↪ evidence, and the mechanism of failure.
3. **Propose a fix** -- describe the change needed (e.g., "add condition `^attr val`",
    ↪ "tighten preference condition", "remove redundant elaboration"). Reference the
    ↪ relevant production file and line if available.
4. **Do not apply the fix** unless the user explicitly asks. The goal of this agent is
    ↪ diagnosis, not modification. (Other agents have skills to apply fixes once the
    ↪ issue is understood.)

## Shutdown

When the investigation is complete, call `soar_shutdown()` to cleanly terminate the
    ↪ environment process. Do not leave the environment running after the session ends.

If you need to re-run the agent with a different seed or after applying a tentative fix,
    ↪ prefer `restart_environment(seed=<seed>)` over `soar_shutdown()` -- it relaunches
    ↪ the environment and reconnects in one step, keeping the broker alive for the next
    ↪ run.

## Key Constraints

- **Port 12121 is fixed** -- the Soar kernel always listens on this port when started via
    ↪ pysoarlib. Do not attempt to change it.
- **Single agent** -- `GetAgentByIndex(0)` is used; all MCP tools target the one agent in
    ↪ the kernel.
- **Do not call `kernel.Shutdown()` directly** -- use `soar_shutdown()` or
    ↪ `soar_execute_command("exit")` only. Calling `Shutdown` on a remote handle crashes
    ↪ the environment.
- **`start_running=False`** -- the agent does not auto-run in MCP mode. All execution is
    ↪ driven by your `run` commands.
````

Analyze Runtime Logs Skill

Soar produces logs of runs that can be processed and analyzed offline. Here we list an excerpt of the instructions that define a skill for analyzing Soar runtime logs that allows various coding model agents (such as the Debugging subagent) to analyze Soar logs. This skill is supplemented with various Python scripts (not included here) that perform various types of text processing on the logs, as summarized in the listing.

```
---
name: analyze-soar-run-logs
description: Tools for analyzing Soar execution log files. Use this skill when examining
    ↪ Soar run logs to extract patterns, diagnose issues, track production firings,
    ↪ monitor operator selection, analyze decision cycles, or compare execution traces.
    ↪ Provides scripts and patterns for systematic log analysis during debugging and
    ↪ validation.
license: Proprietary. LICENSE.txt has complete terms
---

# Analyze Soar Run Logs

## Overview

This skill provides tools and patterns for analyzing Soar execution log files generated
    ↪ during agent runs. Log analysis is essential for debugging issues, validating
    ↪ refactoring changes, understanding agent behavior, and identifying performance
    ↪ bottlenecks (for optimization).

## When to Use This Skill

Use this skill when you need to:

- **Debug unexpected behavior** - Trace decision cycles where agent behavior diverges
    ↪ from expected
- **Validate refactoring** - Compare logs before/after refactoring to ensure identical
    ↪ behavior
- **Identify performance issues** - Find production firing bottlenecks or excessive
    ↪ elaboration cycles
- **Understand operator selection** - Trace why specific operators were/weren't selected
- **Track WME changes** - Monitor creation/removal of specific working memory elements
- **Compare execution traces** - Diff logs from different runs or configurations
- **Extract metrics** - Count decision cycles, operator applications, production firings

## Log File Structure

Soar log files contain structured output controlled by watch levels and debug commands:

### Watch Level Output

- **Watch 0**: Minimal (startup/shutdown messages)
- **Watch 1**: Decision cycles and phases
- **Watch 2**: Operator selection details
- **Watch 3**: Working memory changes (WME additions/removals)
- **Watch 4**: Production firings
- **Watch 5**: Maximum detail (all of the above plus internal details)

### Common Log Patterns

```
Decision Cycle N
```
```

```
  Phase: input
  Phase: proposal
  Phase: decision
    Operator selected: O1
  Phase: application
  Phase: output
```

Production Firing Format

```
Firing: production-name
  Matched: <bindings>
  Actions: <WME additions/modifications>
```

WME Change Format

```
  => WME: (<id> ^attribute value)   # Addition
  <= WME: (<id> ^attribute value)   # Removal
```

Analysis Scripts

The following scripts are available in the `scripts/` directory for automated log analysis:

Available Scripts (Placeholders)

1. **`extract_decision_cycles.py`**
   - Extracts decision cycle boundaries and phase transitions
   - Outputs: CSV with cycle numbers, phases, timing

2. **`count_production_firings.py`**
   - Counts firing frequency for each production
   - Outputs: Sorted list of productions by firing count

3. **`trace_operator_selection.py`**
   - Tracks which operators were proposed and selected each cycle
   - Outputs: Timeline of operator selection decisions

4. **`find_wme_pattern.py`**
   - Searches for specific WME patterns across the log
   - Outputs: Decision cycles where pattern appears

5. **`compare_logs.py`**
   - Diffs two log files to identify behavioral differences
   - Outputs: First decision cycle where logs diverge

6. **`extract_impasses.py`**
   - Identifies impasse creation and resolution
   - Outputs: Impasse types, durations, resolutions

7. **`analyze_elaboration_cycles.py`**
   - Tracks elaboration phase performance
   - Outputs: Cycles with excessive elaborations, bottleneck productions

8. **`extract_production_matches.py`**
   - Parses `production matches` command output

```
   - Outputs: Why productions did/didn't fire

9. **`track_preference_changes.py`**
   - Monitors operator preference evolution
   - Outputs: Preference changes that affected selection

10. **`generate_execution_summary.py`**
    - Creates high-level summary of entire run
    - Outputs: Total cycles, operators used, production firings, final state

### Script Usage Pattern

```bash
# Generic usage pattern (will be implemented)
python .github/skills/analyze-soar-run-logs/scripts/<script-name>.py <logfile> [options]

# Example (future):
python .github/skills/analyze-soar-run-logs/scripts/count_production_firings.py \
    logs/run-2026-01-27.log.txt \
    --top 20 \
    --output firings.csv
```

Manual Analysis Patterns

When scripts are unavailable or insufficient, use these manual patterns:

Pattern 1: Find First Divergence Point

When comparing two logs to find where behavior differs:

1. **Search for decision cycle markers**: Look for "Decision Cycle N"
2. **Compare operator selections**: Check "Operator selected" lines
3. **Identify first mismatch**: Note the cycle number where they differ
4. **Examine state before divergence**: Look at WMEs created in previous cycles

Pattern 2: Trace Production Firing

To understand why a production fired (or didn't fire):

1. **Locate target decision cycle**: Find "Decision Cycle N"
2. **Find production firing**: Search for "Firing: production-name"
3. **Examine bindings**: Check what variables matched
4. **Verify conditions**: Confirm all tested WMEs existed

Pattern 3: Diagnose Missing WME

When expected WME doesn't appear:

1. **Search for creation pattern**: Look for `=> WME: (<id> ^attribute value)`
2. **If not found**: Check if production that should create it fired
3. **Check production matches**: Look for `production matches` output
4. **Verify conditions**: Ensure all conditions for that production were met

Pattern 4: Identify Operator Selection Issue

When wrong operator is selected:

1. **Find decision phase**: Locate "Phase: decision" in target cycle
2. **Check proposed operators**: Look at "Phase: proposal" output
```

```
3. **Examine preferences**: Search for preference output before selection
4. **Trace preference rules**: Find which rules added better/worse/best preferences

### Pattern 5: Performance Analysis

To find production firing bottlenecks:

1. **Extract firing counts**: Look for "production firing-counts" output
2. **Identify high-frequency productions**: Productions firing hundreds of times
3. **Check elaboration cycles**: Cycles with many elaboration firings
4. **Locate infinite elaboration**: Multiple elaborations firing in same cycle repeatedly

## Common Analysis Tasks

### Task: Validate Refactoring

**Goal**: Ensure agent behavior is identical after refactoring

**Approach**:
1. Generate baseline log with seed: `python script.py debug baseline.txt 1234`
2. Apply refactoring changes
3. Generate post-refactoring log: `python script.py debug refactored.txt 1234`
4. Compare logs:
   - Check total decision cycles (should match)
   - Compare operator selection sequence
   - Verify same output commands issued
   - Confirm same final state reached

**Success Criteria**: Logs show identical operator sequences and final outcomes

### Task: Debug Missing Elaboration

**Goal**: Find why expected WME is not created

**Approach**:
1. Identify which production should create the WME
2. Search log for that production's firing
3. If not firing, create debug config with `production matches <name>`
4. Run with debug config to see why conditions aren't met
5. Trace back to find which prerequisite WME is missing

### Task: Debug Operator Selection

**Goal**: Understand why unexpected operator was selected

**Approach**:
1. Find decision cycle with wrong selection
2. Look at all proposed operators in that cycle
3. Check preference output (use `preferences <s> ^operator` in debug config)
4. Identify which preference rule caused selection
5. Examine why preference rule fired (or didn't fire for expected operator)

### Task: Identify Performance Bottleneck

**Goal**: Find productions causing excessive firings

**Approach**:
1. Run with `watch 4` to capture all firings
2. Extract `production firing-counts -f` from end of log
3. Identify productions with highest counts
```

````
4. Examine those productions for optimization opportunities:
   - Can conditions be more specific?
   - Are there duplicate elaborations?
   - Can expensive tests be moved to earlier conditions?

## Integration with Debug Configurations

Log analysis works best when combined with targeted debug configurations:

### Debug Config for Production Analysis

```
production-debug.log.txt
[INITIAL]
watch 4
pwatch --add-filter problematic*production*

[LOOP]
production matches problematic*production*pattern
run -e 1

[END]
production firing-counts -f
```

### Debug Config for Operator Tracking

```
operator-debug.log.txt
[INITIAL]
watch 3

[LOOP]
print --depth 1 <s> ^operator
preferences <s> ^operator
run -d 1

[END]
stats
```

### Debug Config for WME Monitoring

```
wme-debug.log.txt
[INITIAL]
watch 5

[LOOP]
print --depth 5 <s> ^target-attribute
run -e 1

[END]
print --depth 10 <s>
```

## Best Practices

1. **Use deterministic seeds** - Same seed produces same log for reliable comparison
````

```
2. **Create focused debug configs** - Don't try to capture everything; target specific
    ↪ issues
3. **Save and label logs** - Use descriptive names: `baseline-seed1234-2026-01-27.log`
4. **Compare incrementally** - When making changes, compare each iteration
5. **Automate where possible** - Use scripts for repetitive analysis tasks
6. **Document findings** - Keep issue log with relevant cycle numbers and observations
7. **Start broad, then narrow** - Use watch 3 first, then watch 5 for specific cycles
8. **Clean old logs** - Archive or remove logs after issue is resolved

## Log Analysis Workflow

### Step 1: Initial Review
- Check total decision cycles
- Look at final state
- Scan for obvious errors or impasses

### Step 2: Identify Problem Region
- Find approximate decision cycle where issue manifests
- Note what state changes happen around that cycle

### Step 3: Create Focused Debug Config
- Target the problem region with specific watch settings
- Add relevant print statements for state inspection

### Step 4: Analyze Detailed Trace
- Run with focused config
- Extract patterns using scripts or manual search
- Form hypotheses about root cause

### Step 5: Verify Hypothesis
- Create even more focused debug config to test hypothesis
- Run and analyze
- Iterate until root cause confirmed

## Future Enhancements

The following capabilities are planned for future development:

- **Interactive log viewer** - GUI for navigating decision cycles
- **Diff visualization** - Side-by-side log comparison with highlighting
- **Pattern library** - Pre-built patterns for common debugging scenarios
- **Performance profiler** - Automated bottleneck identification
- **Regression testing** - Automated comparison of baseline vs. current logs
- **Log compression** - Extract only relevant cycles for large logs

## Example: Complete Analysis Session

'''bash
# 1. Generate initial log with issue
cd mastermind-example-refactor
python MasterMind-Environment-PySoarLib.py debug 1234
# Issue: Agent selects wrong operator at cycle 15

# 2. Create focused debug config
cat > Soar_Debugger_Specialist/issue-cycle15.txt << EOF
issue-cycle15.log.txt
[INITIAL]
watch 3

[LOOP]
```

```
print --depth 3 <s> ^operator
preferences <s> ^operator
run -d 1

[END]
stats
EOF

# 3. Run focused debug
python MasterMind-Environment-PySoarLib.py debug issue-cycle15.txt 1234

# 4. Analyze log (future - using script)
python ../.github/skills/analyze-soar-run-logs/scripts/trace_operator_selection.py \
    logs/issue-cycle15.log.txt \
    --cycle 15

# 5. Extract preference info manually
grep -A 10 "Decision Cycle 15" logs/issue-cycle15.log.txt
grep "preferences" logs/issue-cycle15.log.txt

# 6. Identify root cause from output
# Found: Missing better preference for expected operator

# 7. Fix and validate
# ... make fix ...
python MasterMind-Environment-PySoarLib.py debug 1234
# Success: Operator selection now correct
```

Related Skills

- **soar-style-conventions** - For understanding production naming patterns in logs
- **create-elaboration-productions** - For debugging elaboration firing issues
- **create-operator-productions** - For debugging operator proposal/application issues
- **create-preference-productions** - For debugging operator selection issues

Scripts Directory

All analysis scripts are located in:
```
.github/skills/analyze-soar-run-logs/scripts/
```

Each script includes:
- Usage documentation
- Example invocations
- Output format description

See individual script files for detailed documentation (to be implemented).

Debugging Trace

Below is the full text of the Opus 4.5 debugging trace summarized in the text.

```
User: The Soar mastermind agent (refactor version) is a work in progress. The agent
    ↪ currently attempts to solve puzzles via a simple approach: it guesses randomly and
    ↪ anytime it observes feedback with 0 black pegs, it marks those choices for those
    ↪ pegs as eliminated for future guesses. (This is not efficient but it should
    ↪ generally work given sufficient turns.) The Soar agent currently solves some
    ↪ problems correctly and other it fails by issuing a duplicate command. This failure
    ↪ is ok given the state of development.  However, in some cases, the agent gets
    ↪ stuck and just runs until an arbitrary exit criterion (150 decisions is met). Your
    ↪ role is to identify the source of this specific "getting-stuck" error and attempt
    ↪ to repair it.

GitHub Copilot: Before starting the debug process, I have a few clarifying questions:

1. **Previous debugging state**: I see logs from earlier debugging on seed 1234 which had
    ↪ a similar "stuck" issue, and notes about a "flawed fix" being applied. Should I
    ↪ check if that fix is still in place in _last-loaded_debug-agent.soar or
    ↪ elaborations.soar? Do you want me to start from a clean state, or work with
    ↪ whatever is currently there?

2. **Debug infrastructure**: Should I use the existing debug configuration system
    ↪ (creating files in `Soar_Debugger_Specialist/`, generating logs) to document my
    ↪ analysis, or do you prefer I just diagnose and fix directly?

3. **Verification scope**: For testing the fix, should I verify against just the seeds
    ↪ you listed, or also test additional seeds to ensure broader correctness?

User: Q1: Work with what is there. The previous changes were incomplete/erroneous and
    ↪ were backed out. Ignore that.
Q2. You are the Soar Issue Debugger (now fixed). Use that infrastructure.
Q3. Use your judgment, but it should be tested against at least these seeds to claim a
    ↪ fix.

Good. Now let me create a debug configuration file and run the baseline tests:

Created [](file:///repo/mastermind/Soar_Debugger_Specialist/issue-stuck-baseline.txt)

Ran terminal command:  for seed in 1118 1119 1111 1113 1112 1114; do echo "=== Seed $seed
    ↪ ===" && python MasterMind-Environment-PySoarLib.py debug issue-stuck-baseline.txt
    ↪ $seed 2>&1 | grep -E "decisions|Game over|Game won|Duplicate|turn_count"; done

Ran terminal command:  python MasterMind-Environment-PySoarLib.py debug
    ↪ issue-stuck-baseline.txt 1118 2>&1 | tail -30

I need to set up the environment first:

Ran terminal command:  source ../setup-env-vars.sh && python
    ↪ MasterMind-Environment-PySoarLib.py debug issue-stuck-baseline.txt 1118 2>&1 |
    ↪ tail -30

Good, seed 1118 works correctly with 70 decisions. Let me run all the tests:

Ran terminal command:  source ../setup-env-vars.sh && for seed in 1118 1119 1111 1113
    ↪ 1112 1114; do echo "=== Seed $seed ===" && python
    ↪ MasterMind-Environment-PySoarLib.py debug issue-stuck-baseline.txt $seed 2>&1 |
    ↪ grep -E "decisions|Game over|GAME|Duplicate|turn_count|Game won"; done
```

```
Seed 1119 is the stuck case. Let me create a detailed debug config to analyze it:

Created [](file:///repo/mastermind/Soar_Debugger_Specialist/issue-1119-detailed.txt)

Ran terminal command:  source ../setup-env-vars.sh && python
    ↪ MasterMind-Environment-PySoarLib.py debug issue-1119-detailed.txt 1119 >
    ↪ logs/issue-1119-detailed.log.txt 2>&1

Ran terminal command:  grep "==>S:" logs/issue-1119-detailed.log.txt | head -5

Found it! State no-change impasse starting at decision 62. Let me examine what's
    ↪ happening just before the impasse:

Ran terminal command:  grep -B5 "^    62:" logs/issue-1119-detailed.log.txt | head -20

Ran terminal command:  grep -A30 "^    61:" logs/issue-1119-detailed.log.txt | head -40

So at decision 61, `guess-peg` fires, but then no operators are proposed at decision 62.
    ↪ Let me look at the operator selections to see how many pegs were guessed before
    ↪ this happened:

Ran terminal command:  grep "O:.*(" logs/issue-1119-detailed.log.txt | tail -30

At decision 61, it's the 3rd guess-peg in this turn. Let me look at the state right
    ↪ before the impasse to see what's happening with peg 4:

Ran terminal command:  grep -E "^    (58|59|60|61|62):" logs/issue-1119-detailed.log.txt

So turn starts at decision 58, and we get 3 guess-peg operators (59, 60, 61) but then no
    ↪ 4th one. Let me check the operator preferences at decision 62:

Ran terminal command:  tail -50 logs/issue-1119-detailed.log.txt

Same pattern as the seed 1234 issue! Two guess-peg operators are proposed for position 4
    ↪ (O181 and O182), but both have BOTH acceptable (+) and reject (-) preferences. The
    ↪ rejects cancel them out, causing a state no-change impasse.

Let me create a focused debug to see the state at decision 61:

Created [](file:///repo/mastermind/Soar_Debugger_Specialist/issue-1119-focus.txt)

Ran terminal command:  source ../setup-env-vars.sh && python
    ↪ MasterMind-Environment-PySoarLib.py debug issue-1119-focus.txt 1119 2>&1 | head
    ↪ -200 | grep -A100 "^print"

Now I can see the situation clearly! At turn 9:
- Position 1: guessed-value 6
- Position 2: eliminated 1,3,4,6 - remaining: 2,5
- Position 3: eliminated 1,2,3,4 - remaining: 5,6
- Position 4: eliminated 1,4,5,6 - remaining: 2,3

The proposed operators are:
- O181 (position 3, value 5)
- O182 (position 3, value 6)
- O179 (position 2, value 2)
- O180 (position 2, value 5)
- O183 (position 4, value 2)
- O184 (position 4, value 3)
```

```
After decisions 59-61, positions 2 and 3 get guessed. Let me see exactly what happens -
    ↪ position 4 should still have valid options (2 and 3). Let me run to decision 62 to
    ↪ see:

Ran terminal command:  source ../setup-env-vars.sh && python
    ↪ MasterMind-Environment-PySoarLib.py debug issue-1119-focus.txt 1119 >
    ↪ logs/issue-1119-focus.log.txt 2>&1 && grep -A80 "^    62:"
    ↪ logs/issue-1119-focus.log.txt | head -90

Let me see the actual state at decision 61 and the preferences:

Ran terminal command:  grep -A100 "^    61:" logs/issue-1119-focus.log.txt | head -120

Now I see the issue! The problem is different from seed 1234. At decision 61:

1. `apply*guess-peg` fires and adds `^guessed-value 3` to P47 (position 4)
2. Because P47 now has a `^guessed-value`, the proposals for O183 and O184 **retract**
    ↪ (they require `-^guessed-value`)
3. All 4 pegs now have guessed-values, so no more guess-peg operators can be proposed
4. The `guess-complete` elaboration should fire and then `send-guess-to-env` should be
    ↪ proposed

But the `send-guess-to-env` operator is NOT being proposed! Let me check the elaboration
    ↪ and proposal rules:

GitHub Copilot: Read
    ↪ [](file:///repo/mastermind/mastermind/elaborations/elaborations.soar#50-50), lines
    ↪ 50 to 65

Read [](file:///repo/mastermind/mastermind/send-guess-to-env.soar#1-1), lines 1 to 40

So `send-guess-to-env` requires `^guess-complete true` and `-^guess-sent`. The
    ↪ elaboration `guess-complete` should fire when all pegs have a guessed-value. Let
    ↪ me check if the `^guess-complete true` is being elaborated. Let me look at what's
    ↪ happening at decision 62:

Ran terminal command:  grep -E "guess-complete|elaborate.*guess"
    ↪ logs/issue-1119-focus.log.txt | head -20

No matches. Let me search for what's happening with the current turn:

Ran terminal command:  grep "guess-complete" logs/issue-1119-focus.log.txt

The `guess-complete` elaboration is NOT firing at all. Let me check whether maybe there's
    ↪ an issue with the peg structure. Let me look at the print output right after
    ↪ decision 61:

Ran terminal command:  grep -A150 "^print --depth 8 s1" logs/issue-1119-focus.log.txt |
    ↪ grep -A60 "^(S1"

Summarized conversation history**CRITICAL FINDING!** Looking at T9's guess (G18), I see
    ↪ the problem clearly now:

**Current state at Turn 9 (T9):**
```
(G18 ^pegs P43)
  (P43 ^peg P44 ^peg P45 ^peg P46 ^peg P47)
    (P44 ^guessed-value 6 ^position 1) # Position 1 has guessed-value
    (P45 ^guessed-value 5 ^position 2) # Position 2 has guessed-value
    (P47 ^guessed-value 3 ^position 4) # Position 4 has guessed-value
```
```

```
    # P46 (position 3) does NOT have guessed-value - that's what operators O181, O182 are
        ↪ proposing for!
```

**But wait - there are only 3 pegs listed with guessed-values (P44, P45, P47). P46
    ↪ (position 3) has NO guessed-value!**

And the operators are proposing for position 3! So the issue is **NOT** that
    ↪ guess-complete can't fire because there are missing guessed-values - the issue is
    ↪ that only 3 of 4 pegs have guessed-values, and we still have operators trying to
    ↪ fill position 3.

Let me look more carefully at decision 61 to see what happens:

Ran terminal command: grep -B5 "^   62:" logs/issue-1119-focus.log.txt | head -30

So at decision 62, we go from decision 61 directly to an impasse. Let me see what
    ↪ happened in decision 61:

Ran terminal command: grep -A50 "^   61:" logs/issue-1119-focus.log.txt | head -60

I see! At decision 61:
1. `apply*guess-peg` fires and adds `^guessed-value 3` to P47 (position 4)
2. The guess-peg operators for position 4 (O183, O184) immediately retract (correct
    ↪ behavior - position 4 now has a value)

But wait - the print shows that position 3 (P46) still doesn't have a guessed-value, and
    ↪ O181/O182 are proposing for position 3. Let me see if they also retract:

Ran terminal command: grep -A100 "^   61:" logs/issue-1119-focus.log.txt | grep -E
    ↪ "O181|O182|position 3|P46"

So O181/O182 (position 3 operators) only have their preferences to O183/O184 retracted.
    ↪ Let me see if O181/O182 themselves get retracted:

Ran terminal command: grep -A120 "^   61:" logs/issue-1119-focus.log.txt | head -130

Now I see the full picture:
1. Decision 61: O184 (position 4, value 3) is selected and applied
2. P47 gets `^guessed-value 3`
3. O183/O184 (position 4 operators) retract correctly
4. Preferences for O181/O182 to O183/O184 retract
5. **BUT O181 and O182 (position 3 operators) are NOT shown retracting!**
6. Then decision 62 has NO operators in the Propose Phase and goes into impasse

Wait, but the print shows position 3 (P46) doesn't have a guessed-value. So O181/O182
    ↪ should STILL be proposed. Why aren't they showing?

Let me search for when O181/O182 retract:

Ran terminal command: grep "O181\|O182" logs/issue-1119-focus.log.txt | tail -30

Interesting! The last entries show:
- `O181 (guess-peg) -  :I` - This is a **reject** preference, not a retraction!
- `O182 (guess-peg) -  :I` - Same here!

O181 and O182 are being **rejected**, not retracted! That's why there are no operators to
    ↪ select - both position 3 operators are being rejected.

Let me look at when and why they get rejected:

```
Ran terminal command:  grep -B3 "O181 (guess-peg) -  :I" logs/issue-1119-focus.log.txt |
    ↪ head -20

Both O181 and O182 are being REJECTED! Let me find what rule is firing these rejects:

Ran terminal command:  grep "reject-duplicate" logs/issue-1119-focus.log.txt | tail -20

`preference*guess-peg*reject-duplicate` is firing and rejecting O181! Let me look at that
    ↪ rule:

Searched for regex `preference\*guess-peg\*reject-duplicate` (`**/*.soar`), 3 results

Read [](file:///repo/mastermind/mastermind/guess-peg.soar#50-50), lines 50 to 75

**FOUND THE BUG!**

Look at `preference*guess-peg*reject-duplicate`:
```soar
sp {preference*guess-peg*reject-duplicate
   (state <s> ^operator <o> +
              ^turns.current-turn.guess.pegs.peg.guessed-value <val>)
   (<o> ^name guess-peg
        ^value <val>)
-->
   (<s> ^operator <o> -)
}
```

This rule rejects any `guess-peg` operator whose value already exists as a
    ↪ `guessed-value` on **any** peg in the current turn.

At turn 9 (T9):
- P44 (pos 1): `^guessed-value 6`
- P45 (pos 2): `^guessed-value 5`
- P47 (pos 4): `^guessed-value 3`
- P46 (pos 3): NO guessed-value yet

The position 3 operators are:
- O181: value **5**
- O182: value **6**

Both 5 and 6 are already used as guessed-values on other positions (P45 and P44)! So
    ↪ `preference*guess-peg*reject-duplicate` correctly rejects BOTH operators.

**The problem is that position 3 has ALL its non-eliminated values (5 and 6) already used
    ↪ elsewhere in the guess!**

Looking at position 3 (P46):
```
(P46 ^eliminated 4 ^eliminated 2 ^eliminated 3 ^eliminated 1 ^position 3 ^status unknown)
```

Position 3 has eliminated values: 1, 2, 3, 4. That leaves only **5 and 6** available.
But 5 is already guessed at position 2, and 6 is already guessed at position 1.

This is a **dead-end situation** - the agent cannot make a valid guess because all
    ↪ available values for position 3 are duplicates of already-guessed values.

The agent needs either:
```

```
1. A way to backtrack and change an earlier peg's guessed-value
2. A fallback mechanism that allows duplicate values in this dead-end case
3. Recognition that this is an unsolvable constraint situation

Let me check if earlier versions had any handling for this, or if the datamap mentions
    ↪ anything about this:

Read [](file:///repo/mastermind/mastermind/datamap.soar)

The datamap doesn't address this edge case. Let me look at the
    ↪ eliminate-peg-value-option.soar file mentioned in the agent structure to see if
    ↪ there's any relevant logic:

Read [](file:///repo/mastermind/mastermind)

Let me update the todo list and document my findings:

Created 8 todos

## Root Cause Analysis Summary

**Bug:** At seed 1119, Turn 9, the agent reaches a dead-end situation where:
- Position 3 (P46) has eliminated values: 1, 2, 3, 4
- Only values 5 and 6 remain available for position 3
- BUT position 1 already has `^guessed-value 6` and position 2 already has
    ↪ `^guessed-value 5`
- The `preference*guess-peg*reject-duplicate` rule correctly rejects ALL remaining
    ↪ options for position 3
- No operators are proposed --> state no-change impasse

**Fix Strategy Options:**

1. **Allow duplicate values as fallback**: When all non-eliminated values for a position
    ↪ are duplicates, allow the duplicate to be guessed anyway. This keeps the game
    ↪ going.

2. **Backtracking**: Implement a mechanism to change an earlier peg's guessed-value when
    ↪ a dead-end is detected. This is more complex.

3. **Strategic guess ordering**: Modify the guess order to avoid this situation (but this
    ↪ may not always be possible).

Option 1 seems simplest - modify the reject-duplicate rule to NOT reject when there's no
    ↪ other choice. Let me think about how to implement this:

The current rule rejects any guess-peg operator whose value is already used. The fix
    ↪ should NOT reject if that value is the ONLY available value for that position.

Let me check what defines "available" - it would be a value that is NOT eliminated on
    ↪ that peg:

Read [](file:///repo/mastermind/mastermind/guess-peg.soar#1-1), lines 1 to 35

The fix approach: Modify `preference*guess-peg*reject-duplicate` to only reject when
    ↪ there's an alternative non-duplicate value available for that peg.

I'll create a modified version that checks: if the proposed value is already used AND
    ↪ there exists another value for this peg that is NOT eliminated AND NOT already
    ↪ used, then reject. Otherwise, allow it.
```

```
Read [](file:///repo/mastermind/mastermind/_last-loaded_debug-agent.soar)

Now let me implement the fix. The strategy is to modify the reject-duplicate rule to only
    ↪ reject when there's a non-duplicate alternative available:

Using "Replace String in File"


```

```
Now let me test this fix with seed 1119:

Ran terminal command:  source ../setup-env-vars.sh && python
    ↪ MasterMind-Environment-PySoarLib.py debug 1119 2>&1

The fix allowed progress! But we got "Duplicate move detected" because the agent now
    ↪ tries guess `6 5 6 3` - it sent value 6 twice (positions 1 and 3).

The environment rejects this as invalid. So the fix allows the agent to proceed past the
    ↪ dead-end, but it creates an invalid guess that the environment flags as a
    ↪ duplicate.

This is expected behavior for the fix - it allows duplicates when there's no other
    ↪ choice. But this particular game doesn't allow duplicates at all.

Let me check if the log shows the game terminated properly:

The key progress:
The key progress:
- **Before fix**: Stuck at decision 62 with state no-change impasse (150 decisions)
- **After fix**: Progresses to decision 63, submits a duplicate guess, game detects
    ↪ duplicate and ends

This is actually correct behavior! The agent's elimination rules have backed it into a
    ↪ corner where the only possible move is a duplicate. The fix allows the agent to
    ↪ try that move, and the environment correctly rejects it.

Let me test all 6 seeds:

Ran terminal command:  source ../setup-env-vars.sh && for seed in 1118 1119 1111 1113
    ↪ 1112 1114; do echo "=== TESTING SEED $seed ===" && python
    ↪ MasterMind-Environment-PySoarLib.py debug $seed 2>&1 | grep -E
    ↪ "decisions|Duplicate|game over|won|halt|Game over|GAME"; done

**Excellent results!** All 6 seeds now terminate properly:

| Seed | Decisions | Outcome              | Notes                  |
|------|-----------|----------------------|------------------------|
| 1118 | 70 | Duplicate (repeats guess)  | Terminates properly   |
| 1119 | 63 | Duplicate (sends 6 5 6 3)  | Fixed! Was stuck at 150 |
| 1111 | 70 | Duplicate (repeats guess)  | Terminates properly   |
| 1113 | 70 | Duplicate (repeats guess)  | Terminates properly   |
| 1112 | 70 | **WON!**                   | Working correctly     |
| 1114 | 119 | Duplicate (repeats guess) | Terminates properly   |

The fix works:
- **Seed 1119** now terminates at 63 decisions instead of getting stuck at 150
- **Seed 1112** wins the game
- All other seeds terminate via "duplicate detected" which is expected game exit behavior
```

```
Now let me update the todo list and apply the permanent fix:

Starting: *Apply permanent fix* (8/8)

Now let me apply the permanent fix to the main guess-peg.soar file:

Read [](file:///repo/mastermind/mastermind/guess-peg.soar#45-45), lines 45 to 70

Using "Replace String in File"


```

```
Now clean up the debug file by removing the duplicate fix (since it's now in the main
    ↪ file):

Using "Replace String in File"

Read [](file:///repo/mastermind/mastermind/_last-loaded_debug-agent.soar)

Using "Replace String in File"

Let me try a different approach:

Created [](file:///repo/mastermind/mastermind/_last-loaded_debug-agent.soar)

Ran terminal command:  cat > /repo/mastermind/mastermind/_last-loaded_debug-agent.soar <<
    ↪ 'EOF'
# Debugging productions for monitoring operator ties

# Monitor and print tied operators when an operator tie impasse occurs
sp {monitor*operator-tie*print-candidates
   (state <s> ^impasse tie
              ^attribute operator)
   -->
   (write (crlf) |=== OPERATOR TIE DETECTED ===|)
   (write (crlf) |Tied operator candidates:|)
   (write (crlf) (cmd print --depth 5 <s>))
}
EOF

Now let me run a final validation with all 6 seeds to confirm the permanent fix works:

Ran terminal command:  source ../setup-env-vars.sh && for seed in 1118 1119 1111 1113
    ↪ 1112 1114; do echo "=== SEED $seed ===" && python
    ↪ MasterMind-Environment-PySoarLib.py debug $seed 2>&1 | grep -E
    ↪ "decisions|Duplicate|game over|won|halt|Game over|GAME|excised"; done

**All tests pass!** No productions are excised now (the permanent fix is in place). Let
    ↪ me update the todo list and create the final documentation:

Completed: *Apply permanent fix* (8/8)

Now let me create the issue documentation:

Created
    ↪ [](file:///repo/mastermind/Soar_Debugger_Specialist/debugging-logs/issue-stuck-dead-end-duplicate-2025
```

```
## Summary

**Bug Fixed Successfully!**

### Root Cause
The `preference*guess-peg*reject-duplicate` rule in guess-peg.soar was unconditionally
    ↪ rejecting any `guess-peg` operator whose value was already used elsewhere in the
    ↪ current guess. At seed 1119 turn 9, the agent reached a dead-end where position 3
    ↪ had only values 5 and 6 available (all others eliminated), but both were already
    ↪ used in other positions. All operators were rejected, causing a state no-change
    ↪ impasse.

### Fix
Modified the rule to only reject when a non-duplicate alternative exists. If all
    ↪ available values for a peg are already duplicates, the agent now allows the
    ↪ duplicate guess to proceed. The environment then properly detects and rejects the
    ↪ invalid move, allowing the game to terminate gracefully.

### Validation
All 6 test seeds now terminate properly:
- **Seed 1119**: Fixed from 150 decisions (stuck) to 63 decisions (duplicate detected)
- **Seed 1112**: Still wins correctly
- **All others**: Terminate via duplicate detection as expected

Made changes.
```

Acronyms

DAP Debug Adapter Protocol. 9, 10

GUI graphical user interface. 15

IDE integrated development environment. 2, 4

LLM Large Language Model. 2, 4, 12, 16

LSP Language Server Protocol. 9, 15

MCP Model Context Protocol. 8–11

SE software engineering. 4

SML Soar Markup Language. 10